



Sarajevo School of
Science and Technology

Sarajevo School of Science and Technology
Department of Computer Science

**Sim-to-Real Transfer of Reinforcement Learning
Policies for an Autonomous Miniature RC Racing Car**
Undergraduate thesis

Mentor: Alen Bošnjaković

Student name: Nijaz Andelić

Sarajevo, May 2026

Dedications and Acknowledgements

This thesis is dedicated first and foremost to my mother and father, Ehlimana and Nedim, who invested their lives into mine and sacrificed so much of their own comfort, success, and careers for my future. Everything I have achieved is built on their love, patience, and belief in me. Thank you for everything. This work is for you.

I also dedicate this thesis to my Mini, who quietly stepped into my heart and decided to stay there forever. Thank you for being my constant companion, my source of comfort, and for staying beside me through every difficult moment. Love you!

I really owe a special thanks to my mentor, Professor Edin Fazlić. His kindness, patience, understanding, guidance, and mentorship had a major influence on this thesis, on my development as a student, and on my development as a person. I can honestly say that I am a better person because of Professor Edin Fazlic. Thank you for your support, understanding, and belief in this project.

A Note of Declaration

I, Nijaz Andelić, hereby declare that the thesis entitled “Sim-to-Real Transfer of Reinforcement Learning Policies for an Autonomous Miniature RC Racing Car” is my original work, and that I have not copied from any other work or source except where reference or acknowledgement is made in the text. The views expressed in this work are my own, and no part of this thesis has been written for me by another person. I have cited all sources from which I have drawn information appropriately, and this work has not been submitted toward any degree or award at any other university or place of learning.

I further declare that AI tools, including ChatGPT by OpenAI and Gemini by Google, were used only as support tools during the preparation of this thesis, mainly for refining language, improving structure and clarity, reviewing documentation-style explanations, and supporting the presentation of tables, captions, and experimental results. All technical decisions, implementation work, hardware setup, model training, calibration, testing, evaluation, and final interpretation of results were carried out and reviewed by the author. Any AI-generated suggestions were modified where necessary and accepted only when they accurately reflected the actual implementation and results of the project.

Date: _____

Signature: _____

ABSTRACT

This thesis presents the design, implementation, and evaluation of a sim-to-real reinforcement learning pipeline for an autonomous miniature RC racing car. A custom Webots environment was developed to approximate the physical vehicle and indoor track, providing camera-based observations, continuous control actions, Ackerman steering, reward calculation, and episode termination. Two Stable-Baselines3 algorithms were implemented: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). PPO was trained for 1,000,000 timesteps as the main long-run visual policy experiment, while SAC was trained as a comparative continuous-control method. Results show that image-based reinforcement learning learned meaningful driving behavior in simulation. SAC achieved the strongest deterministic Webots evaluation, reaching the configured maximum episode length in most logged episodes during a 10-minute run. Real-world testing confirmed partial transfer under safety constraints, while highlighting remaining challenges caused by lighting, camera angle, wall proximity, actuator calibration, and limited sensing.

LIST OF FIGURES

Figure 1: Final Webots racing environment used for simulation-based reinforcement learning.

Figure 2: Example RGB camera observation used as input to the reinforcement learning agent.

Figure 3: Reinforcement learning training pipeline connecting the Webots simulation environment with the learning algorithm.

Figure 4: Overall architecture of the autonomous miniature racing system.

Figure 5: Physical RC vehicle platform with the OAK-D Lite camera mounted on the front/top section for forward-facing visual perception.

Figure 6: Physical control electronics architecture connecting the Raspberry Pi, Arduino, steering servo, ESC, and external actuator power.

Figure 7: PPO reward design showing boundary detection, road center estimation, dense reward calculation, and terminal penalties.

Figure 8: SAC observation and reward pipeline based on RGB input, distance progress, action penalties, and terminal safety conditions.

Figure 9: Mean episode reward during final PPO training.

Figure 10: Mean episode length during final PPO training.

Figure 11: SAC mean evaluation reward over training timesteps.

Figure 12: SAC mean evaluation episode length over training timesteps.

Figure 13: Rolling SAC steering and throttle behavior during the 10-minute deterministic evaluation.

Figure 14: Episode lengths during 10-minute deterministic Webots evaluation.

Figure 15: Termination reason distribution during deterministic Webots evaluation.

Figure 16: Mean absolute steering command per episode during deterministic Webots evaluation.

Figure 17: Problematic lower chicane crash zone observed during testing.

Figure 18: Problematic track section observed during real-world testing.

Figure 19: Physical indoor racing track used for real-world sim-to-real testing.

LIST OF TABLES

Table 1: Hardware components used in the physical autonomous racing platform.

Table 2: Main software modules used in the project.

Table 3: Comparison of final PPO and SAC training configurations.

LIST OF ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

CNN - Convolutional Neural Network

DDPG - Deep Deterministic Policy Gradient

ESC - Electronic Speed Controller

FPS - Frames Per Second

Gym - OpenAI Gym / Gym-style Reinforcement Learning Interface

HSV - Hue, Saturation, Value

IMU - Inertial Measurement Unit

LiDAR - Light Detection and Ranging

MDP - Markov Decision Process

MPC - Model Predictive Control

OAK-D Lite - OpenCV AI Kit Depth Lite Camera

PPO - Proximal Policy Optimization

PWM - Pulse-Width Modulation

RC - Radio-Controlled

RGB - Red, Green, Blue

RL - Reinforcement Learning

SAC - Soft Actor-Critic

SB3 - Stable-Baselines3

SDE - Stochastic Differential Equation

SLAM - Simultaneous Localization and Mapping

ToF - Time-of-Flight

USB - Universal Serial Bus

VNC - Virtual Network Computing

Contents

ABSTRACT	4
LIST OF FIGURES	5
LIST OF TABLES.....	6
LIST OF ABBREVIATIONS	6
1. INTRODUCTION	9
2. LITERATURE REVIEW	11
2.1 The Challenge of Autonomous Racing.....	11
2.2 Classical Baselines: Model Predictive Control (MPC).....	11
2.3 Deep Reinforcement Learning for Continuous Control.....	12
2.4 The Sim-to-Real Gap	13
2.5 State-of-the-Art Sim-to-Real Transfer	13
2.6 Research Gap and Thesis Positioning.....	14
3. PROBLEM ANALYSIS.....	16
4. BACKGROUND AND REINFORCEMENT LEARNING ALGORITHMS	19
4.1 Reinforcement Learning Overview.....	19
4.2 Proximal Policy Optimization (PPO).....	19
4.3 Soft Actor-Critic (SAC).....	20
5. RESEARCH METHODOLOGY	22
5.1 Simulation Environment	22
5.2 Observation Space	24
5.3 Action Space	26
5.4 Reinforcement Learning Algorithms	26
5.5 Training Pipeline.....	26
5.6 Evaluation Metrics	28
6. SYSTEM DESIGN AND IMPLEMENTATION	30
6.1 Hardware Platform.....	30
6.2 Software Architecture	33
6.3 Real-World Deployment Pipeline.....	35
6.4 Sim-to-Real Transfer Strategy	38
7. EXPERIMENTAL SETUP	40
7.1 PPO Training Configuration	40
7.2 SAC Training Configuration.....	41

7.3 Reward Function Design.....	42
7.4 Evaluation Setup	45
7.5 Transition to Real-World Testing	45
8. RESULTS AND DISCUSSION.....	46
8.1 Simulation Environment Implementation.....	46
8.2 PPO Training and Evaluation Results.....	46
8.3 SAC Training and Evaluation Results	49
8.4 PPO and SAC Comparison	51
8.5 Real-World Deployment Results	53
8.6 Limitations	59
8.7 Discussion	60
9. CONCLUSION	62
10. FUTURE WORK	64
BIBLIOGRAPHY	66

1. INTRODUCTION

Autonomous Vehicles are among the fastest growing segments of Artificial Intelligence and Robotics today. While much of the autonomous vehicle development is focused on safety and reliability in structured environments (such as urban roadways), autonomous racing represents a significantly different problem to solve. Autonomous Racing vehicles typically drive at or very close to their physical limits. Therefore, they require extremely fast sensing of the environment, precise control over steering, acceleration, braking, and continuous optimization of trajectory. The above properties have made autonomous racing an attractive testing ground for high performance robotic control [1].

Reinforcement learning (RL) is one of several machine learning techniques that emerges with great potential to develop complex control strategies for learning how machine can interact with environment, and reinforcement learning has shown promise for the development of autonomous racing control policies. Classically, there are many approaches to model-based control methods including Model Predictive Control (MPC). Both large scale autonomous racing vehicles [3], and small scale autonomous racing models [4], [5] have demonstrated excellent performance using these methodologies. In contrast to MPC based methods which require a complete and accurate model of the vehicle's behavior, RL provides an alternative method to develop complex control policies without the need for a model. The ability to develop control policies by optimizing them through trial and error is what makes RL so appealing for developing control policies for systems that have complex non-linear behaviors like autonomous racing vehicles.

However, there is a significant problem with training policies for simulation-based environments. They typically do not transfer successfully to robotics operating in the "real world." The term sim-to-real gap (also known as the gap between simulator and reality) refers to the fact that no simulator will ever be able to fully replicate the characteristics of the "real world," which may include differences in lighting, sensor noise, friction, actuator latency, etc. Thus, a policy that performs well in simulation may also operate effectively in the "real world", but it could also potentially fail or operate erratically.

This thesis investigates the feasibility of transferring reinforcement learning policies trained in simulation to a physical 1:12 scale RC vehicle equipped with onboard sensing and computation, including a forward-facing RGB camera. The reinforcement learning agent is

trained primarily in a physics-based simulation environment before being deployed to the real vehicle.

The central research question addressed in this thesis is:

How effectively can reinforcement learning policies trained in simulation transfer to autonomous control of a physical miniature RC car?

To answer this question, the project uses a structured process of developing a simulation environment based on Webots (for simulation) to develop and train reinforcement learning policies, evaluating those policies through deterministic simulations, and physically testing them in a safe manner using real-world deployments. The complete system is composed of a Webots-based training environment, policies developed from PPO and SAC reinforcement learning algorithms, log files developed as part of the evaluation of the policies, and a physical deployment pipeline consisting of an OAK-D Lite camera, a Raspberry Pi inference engine, an Arduino-based controller for actuators (steering servo and electric speed controller).

The main contributions of this thesis are:

1. the development of a Webots-based simulation environment for a miniature RC racing vehicle,
2. application and evaluation of PPO and SAC reinforcement learning algorithms with camera-based inputs,
3. a real-world deployment pipeline using OAK-D Lite camera, Raspberry Pi, Arduino, servo, and ESC controls,
4. the analysis of practical sim-to-real transfer limitations observed during physical testing, including lighting sensitivity, wall proximity, camera mounting, and actuator calibration.

2. LITERATURE REVIEW

2.1 The Challenge of Autonomous Racing

The autonomous racing environment represents an extreme test case for all types of highly-agile robots in order to push systems to perceive and control a vehicle under conditions that are as close to the physical limits of performance as possible. The autonomous racing environment differs greatly from typical urban self-driving environments. Urban self-driving typically focuses primarily upon ensuring safe operation and adhering to traffic laws. Racing, however, requires vehicles to operate within a nonlinear dynamic system at high speed. As stated by [1], the automotive industry is currently divided between these two distinct approaches to developing such systems: the traditional modular pipeline (i.e., separate mapping, perception and control) versus the newer end-to-end machine learning approach.

[2] created the F1TENTH platform using an open-source 1:10 scale vehicle that has LiDAR sensor technology and onboard computing. The platform was one of the first and most well-known platforms for testing autonomous racing systems. Today it is often considered a benchmark for assessing autonomous racing methods because of the significant cost savings and reduced risk of damage compared to actual cars, while still allowing researchers to study similar dynamic behaviors as those experienced by the larger version of the vehicle (e.g., sideslip angle and load transfer). For the purposes of this research, both the concept and methodology of scaled autonomous racing are based on the F1TENTH ecosystem. Additionally, the testbed presented here is a modified commercial 1:12 scale radio controlled car.

2.2 Classical Baselines: Model Predictive Control (MPC)

Historically, MPC was considered the most widely used baseline method for Autonomous Racing. MPC approaches use an explicit model of the vehicle's dynamics and then compute optimal steering/throttle inputs through solving a constrained optimization problem at each sampling period. Typically, the goal of MPC approaches is to find the minimum possible lap time given that the vehicle's physical/dynamic constraints are met.

The effectiveness of MPC-based approaches has been clearly demonstrated in practice. [3] showed a successful implementation through the AMZ Driverless team, which won the Formula Student Germany competition. Their system followed a fully modular architecture that combined Simultaneous Localization and Mapping (SLAM) with a high-frequency MPC controller. Comparable results have also been shown in miniaturized race cars. [4] and [5] show that optimization-based control methods can be used with 1:43 scaled automobiles. [5] used off-board processing while [4] has shown that such control contour equations may be run on the hardware itself, at very low computational load.

Although it is highly successful, one major drawback of MPC is the reliance upon an accurate representation of the dynamic behavior of the controlled vehicle. As discussed by [4], if there exists significant error in identifying critical parameters including tire friction coefficient, and/or the moment of inertia about the center of mass, then a good level of accuracy in tracking desired paths may not be possible. Due to factors like temperature variations, contaminated surfaces, tire wear etc., these parameters will never remain constant during actual racing. Thus when the assumed model does not represent the current dynamic behavior of the automobile, a controller based on MPC can degrade rapidly, and even fail. The drawbacks of using MPC have given rise to increased interest in using RL techniques to create control policies directly through experience rather than via a static mathematical model.

2.3 Deep Reinforcement Learning for Continuous Control

Modern continuous control is theoretically based on the Markov decision process (MDP) that was originally developed by [6]. Continuous RL was primarily limited to discrete action spaces until [7] introduced DDPG. [8] then extended this idea to continuous action space, creating a method for deep neural nets to directly produce continuous actions like steering angle and throttle. Building on these developments [9] created proximal policy optimization (PPO) by including a clipper. PPO has become a staple of robotics because it is very simple and very robust. In this thesis, PPO is used as the main long-run visual policy training configuration, while Soft Actor-Critic (SAC) is explored as a comparative continuous-control method.

Recent research has shown that reinforcement learning is effective in achieving success in high-performance racing applications. [10] developed an RL (reinforcement learning) agent within the Gran Turismo Sport simulator with its own high level of fidelity. They also were able to show that their “Super-Human” AI was capable of competing against and ultimately beating the best human driver data from many players. In another similar field of study, [11] used curriculum learning to develop RL (reinforcement learning) agents over time by training them on increasingly harder racing-related tasks which would allow for behaviors including overtaking. Overtaking is particularly hard to implement when you use traditional rule based planning methods.

2.4 The Sim-to-Real Gap

While reinforcement learning works fairly well in simulations, it has been harder to successfully transfer what is learned into a “real” physical environment. This problem is known as the “sim-to-real” gap. The gap arises due to inherent differences between a simulation model and the actual world, such as differences in how objects appear (e.g., different lighting, textures and shadows) to the cameras used for the navigation. Additionally, there can be significant differences in the way the systems behave (due to the lack of accurate modeling of things like friction, motor action and delay associated with actuators). Therefore, while the policy may demonstrate consistent and efficient driving in a simulator, it may become unstable immediately after being deployed onto a real-world platform, possibly demonstrating oscillating behavior or failing completely.

2.5 State-of-the-Art Sim-to-Real Transfer

Recent work has explored a range of transfer learning strategies with the goal of reducing the sim-to-real gap. [12] demonstrated the feasibility of vision-based reinforcement learning by training an agent to control a rally car using raw RGB image inputs. This study is particularly relevant to this thesis, which employs an OAK-D camera, as it shows that convolutional neural networks (CNNs) can extract sufficient state information for high-speed control without relying on costly LiDAR sensors.

The main discussion in recent literature concerns the architectural design of reinforcement learning agents for autonomous racing. [13] addressed this question through a comparative

study conducted on a custom miniature RC racing platform inspired by scaled autonomous racing research, where they evaluated three distinct learning architectures: a Full Planning approach that mirrors the classical modular pipeline, a Trajectory Tracking architecture, and a fully End-to-End model that maps sensor inputs directly to motor commands. Although the planning-based agents achieved the strongest performance in simulation, the end-to-end models demonstrated superior transfer to real-world deployment. [12] attribute this result to the reduced reliance of end-to-end architectures on intermediate state estimations, such as precise vehicle localization, which are particularly sensitive to sensor noise and modeling errors in real environments.

More recent work has also focused on reducing dynamics-related mismatches between simulation and real hardware. [14] reported that a reinforcement learning policy trained entirely in simulation was able to outperform state-of-the-art MPC controllers when deployed on a physical F1TENTH vehicle. A key contribution of their approach was the explicit modeling of actuator dynamics, including steering servo delays and motor response characteristics, rather than focusing solely on tire friction. By incorporating these actuator models into the simulation environment, their work significantly improved policy transfer and reduced the sim-to-real gap.

Beyond improving static simulation accuracy, other approaches have begun to address sim-to-real transfer through online adaptation mechanisms. [15] proposed an approach for aerial robots in which simulation physics parameters are updated online using residual errors observed during real-world deployment. In a related direction, [16] used Neural Stochastic Differential Equations (SDEs) to model highly non-linear drifting behaviors, showing that learned dynamics models can represent highly unstable driving regimes more effectively than purely analytical physics models.

2.6 Research Gap and Thesis Positioning

The current body of research shows that there was clearly a shift in the focus of autonomous racecar development from traditional optimization-based control methods (such as MPC) to learning-based methods using reinforcement learning. Even though MPC remains useful as a stable method with good model visibility, its maximum performance is limited by how accurate the car model is. On the other hand, reinforcement learning has been able to achieve

incredible results, including in many instances “super-human” levels of performance in simulations. However, the quality of these results typically degrade significantly once they are applied in real-world environments due to the sim-to-real gap.

This thesis combines the concepts described in both [13] and [14], where a full end-to-end training methodology is employed so as to minimize the negative impacts of sensor errors and/or inaccuracies in estimating system states. Furthermore, the thesis explicitly includes models of the actuators used during the training process. By employing these methodologies, the research will examine whether or not it is possible to achieve reasonable levels of success with regard to sim-to-real transfer on a small scale RC racing platform that was developed in a similar manner as most miniaturized versions of autonomous racing platforms.

3. PROBLEM ANALYSIS

Autonomous race car control has several difficulties as it combines high speed vehicle dynamics with very limited sensory information and requires quick decisions to achieve the goal of winning. The race car environment also differs from other autonomous vehicles in that the vehicle must make aggressive control moves. Unlike autonomous vehicles, which have a safe operating zone and can assume certain environmental conditions will remain constant, racing vehicles are required to adapt quickly to their own rapidly changing state. The nature of this environment introduces many problems with regards to deploying a reinforcement learning based control system on an actual robot.

One of the main issues here is the “sim to real” gap, which refers to the difference between how the simulated world works for training purposes and how the physical world actually behaves when the policy trained using the simulator is applied to the actual hardware. [12][14]. Although simulators can produce very realistic models of the real world, they do not capture all of the complexities of real-world physics. Surface friction, tire deformation, suspension motion, and aerodynamics can all cause differences between the simulated and real worlds, such that a policy that performed well during training in a simulator could perform poorly (or even fail), once it is run on the real world’s hardware.

A second issue is introduced by a disparity in the type of data obtained by the sensors when they are running in the simulation versus those being used with the actual vehicle. Simulated sensors provide clean, and generally noise-free measurements. However, real sensors produce many different types of uncertainties. As an example, the RGB (red, green, blue) camera installed on the vehicle experiences changes in light levels, motion blur, and image noise which could negatively impact the quality of perception. Another example would be that the simulated camera’s field of view, and resolution may vary from the physical OAK-D Lite camera being used on the actual vehicle. These discrepancies can have the policy based on perceptions perform differently than it did within the simulated environment.

The physical test runs were done with the vehicle near walls of the garage during the garage run. This was far closer to clean vertical walls than were the tracks of the simulation run, and the garage run had a much more confined visual scene than did either the simulated track and the attic setup. It produced a dramatically different input for the cameras. The policy

occasionally interpreted the edge or wall-contrast as part of the line-delineating features of the lane, resulting in a wrong decision made by the steering component.

Actuators' dynamics and control latencies present another significant problem. Majority of simulators use an ideal instant response for modeling vehicle actuation. However, all real-world actuators have inherent delay characteristics and non-linear behaviors. Steering servo's movement will take some time to move from one angle to another. Brushless motors have inherent delay in their response to throttle input along with non-linear throttle characteristics. Thus, these characteristics create control latency and dynamics which may be difficult to accurately model in simulation. This could cause instability in policy training if ideal conditions are used.

Additionally, the limitations on size of the experimental test bed itself creates further limitations. The race track used was 4m x 3m in size with approximately 0.9 m wide lanes. Thus, there is very little room for error in steering maneuvers while attempting to maintain stability through corners. A slight deviation in steering angle or throttle usage can result in contact with the boundary of the track or loss of vehicle control.

In addition, the reinforcement learning algorithm itself causes problems with training stability. As a result, deep reinforcement learning has been found to be highly dependent on rewards, parameter settings, and explorations. Reward misshaping may cause unwanted behavior such as oscillating steering or too much braking. Additionally, training on high-dimensional vision data will dramatically increase the difficulty of training and consequently the number of samples needed for training.

In addition, when using machine-learned policies to control vehicles in the real world, we have to consider all aspects related to safety. High-speed autonomous operation can cause accidents that may damage the vehicle or sensor equipment. Therefore, the testing was done in controlled conditions. This included limited throttle input, manually arming the test system, an emergency stop function and indoor tracks with controlled speed. The controlled conditions enabled validation of the deployment pipeline with low risk of damaging any hardware.

Therefore, we used simulation and a carefully designed reinforcement learning pipeline to address these issues. The project incorporated simulation model vehicle dynamics, camera based observation, actuation limits, and safety features to minimize the “sim-to-real” gap. The last experiments showed that the policy trained in simulation could be transferred to the real car successfully at the controlled-testing level. However, they also demonstrated that successful, reliable and unconstrained racing in real world would require increased robustness, better sensing systems, and additional adaptation to real world.

4. BACKGROUND AND REINFORCEMENT LEARNING ALGORITHMS

4.1 Reinforcement Learning Overview

Reinforcement Learning (RL) is a machine learning paradigm in which an agent interacts with an environment and learns decision-making skills through this interaction. The agent identifies the present condition of the environment, acts upon it and receives a reward signal for how good the act was. Through continued interaction with the environment, the agent develops a policy that will optimize total reward. [6]

A reinforcement learning problem can be modeled using a Markov Decision Process (MDP). A MDP is defined by a set of attributes that include states, actions, transition functions, and rewards. The agent uses these attributes to select an action at each step in time using its current policy. Once the agent has selected an action, the environment then moves into another state.

For autonomous racing, the agent's state is generally described by sensor readings such as RGB images and optionally inertial measurements. An example of an action would be to send the vehicle two continuous control signals, i.e., steering and throttle. Rewards are assigned to encourage progression down the track while discouraging both collision and loss of control. Creating a valid reward function is difficult because if there are problems with defining the reward function, the agent could develop undesirable characteristics such as oscillating steering or local optimums.

4.2 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is one of the most commonly employed Reinforcement Learning algorithms. The purpose of PPO is to provide greater training stability than prior to policy gradient algorithms. Since PPO is an On-Policy Algorithm, the updates to the policy are based upon the data generated from the current policy [9].

The major concept with regards to PPO is preventing too many updates to the policy during training. Standard Policy Gradient Methods can result in unstable training and eventually

collapse. PPO reduces this problem by including a Clipping Mechanism. This method limits each update step to how far the policy will be updated.

The objective function used in PPO is defined as [9]:

$$L^{CLIP}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (1)$$

where $L^{CLIP}(\theta)$ is the clipped PPO objective function, θ represents the policy parameters, $r_t(\theta)$ is the probability ratio between the new policy and the old policy, A_t is the advantage estimate, and ϵ is the clipping parameter. The advantage estimate shows whether the selected action was better or worse than expected. The clipping function limits $r_t(\theta)$ to the interval $[1 - \epsilon, 1 + \epsilon]$, so the new policy cannot move too far away from the old policy during one update.

PPO will be much less likely to create an unstable policy update because this mechanism. This is why PPO has been chosen to be our primary algorithm in the long-term visual policy learning study. In addition to using PPO, we have also used SAC (SAC) as our second algorithm because it represents a sample efficient off-line policy method that can perform well on the continuous control problem.

4.3 Soft Actor-Critic (SAC)

SAC (Soft Actor Critic) is a form of off-policy reinforcement learning for continuous control tasks. However, unlike PPO, SAC uses previous experiences collected by the model, thus making it much more efficient with respect to samples.

One way that SAC differs from other forms of RL algorithms is through the incorporation of an entropic component within the objectives of the model. Alongside attempting to maximize the expected reward, SAC attempts to provide the model with some level of random action, resulting in greater exploration and a reduced risk of prematurely converging to a suboptimal policy.

The objective function of SAC can be stated as follows [17]:

$$J(\pi) = \mathbb{E}[\sum_t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)))] \quad (2)$$

where $J(\pi)$ represents the SAC objective function, π is the policy, s_t is the state at time step t , a_t is the action taken at time step t , and $r(s_t, a_t)$ is the reward received for taking action a_t in state s_t . The term $\mathcal{H}(\pi(\cdot | s_t))$ represents the entropy of the policy, which measures how random or exploratory the policy is in a given state. The parameter α controls how important the entropy term is compared to the reward term.

The use of SAC for exploration purposes may help identify optimal behaviors when those are difficult to determine. It can also learn from experiences that were collected while using other policies. Because of this property (off-policy), it has potential to perform better with respect to sample efficiency compared to many other on-policy algorithms.

In this thesis, SAC is used as a secondary algorithm to compare against PPO and evaluate the impact of different reinforcement learning approaches on autonomous racing performance.

5. RESEARCH METHODOLOGY

This study uses an experimental design as it is used to determine the potential for the transfer of control policies learned through reinforcement learning in simulations to a physical race car that will operate autonomously. The experimental design method will use both simulated testing and ultimately testing of these policies in the real world (i.e., physically) with the first step being the development of a simulation, then the development of reinforcement learning agents to learn from this simulation, and finally to test/evaluate these policies on a small scale racing car.

5.1 Simulation Environment

The initial phase involved the creation of a physics based simulation that mimics the actions of the actual vehicle. This simulation was used as the primary training space for reinforcement learning, and allows for testing and evaluation prior to implementation into the physical system.

Webots Robotics Simulator [19] was chosen as the primary simulation tool due to its ability to create rigid body simulations, include sensors models, and provide an integrated python interface for reinforcement learning. The capabilities of Webots allowed the simulating the vehicle to Gymnasium compatible reinforcement learning framework and Stable Baseline3 [18] for both PPO and SAC training.

Digital versions of the RC vehicles were created in Webots with measured and estimated values from the physical platform. The digital versions contained the major geometric characteristics of the platform, drivetrain, wheelbase, track width, tire characteristics, approximate weight distribution, and steering and motor interfaces. These values allow the simulated version of the vehicle to replicate the physical systems dynamic responses when accelerating, braking, or turning.

The vehicle model incorporates the following physical characteristics:

- overall vehicle dimensions,
- wheelbase and track width,
- tire radius and width,
- suspension-related behavior,

- drivetrain configuration,
- approximate vehicle mass and center-of-mass location,
- Ackermann-style steering behavior,
- motor velocity control.

The race track within the simulation environment had been created for the purpose of conducting reinforcement learning research. A racing track had initially been designed and tested in Webots. Then a physical version of the race track was created to be visually close to the simulation design. The last test using the compact indoor race track included a light colored road, white edge lines, and red and blue boundary areas. The visual similarity of this real configuration to the simulation provided a better model for a physically developed prototype that would allow a practical solution to develop indoors.

The width of the lane where the vehicle could drive was approximately 90 cm, providing a suitable size for the miniature RC car while requiring precision steering. Due to its compact size, there existed many difficulties in completing the task due to the limited amount of time available for correcting steering or throttle inputs before a collision with one of the red and blue boundary areas occurred.

The red and blue boundary colors were chosen for practical and algorithmic purposes. Practically speaking, they offer high contrast compared to the black driving area and white edge marks. From an algorithmic standpoint, it is easy to find the red and blue colors in HSV (Hue Saturation Value) color space. In addition, having two different colors allowed for the visual policy to identify each side of the race track within the simulated camera view. This proved very valuable since the agent was relying upon RGB data as opposed to an explicit map or location information. Red and Blue boundary areas were also applied to the physical version of the race track. Therefore, the real world visual environment was much closer to the simulation.

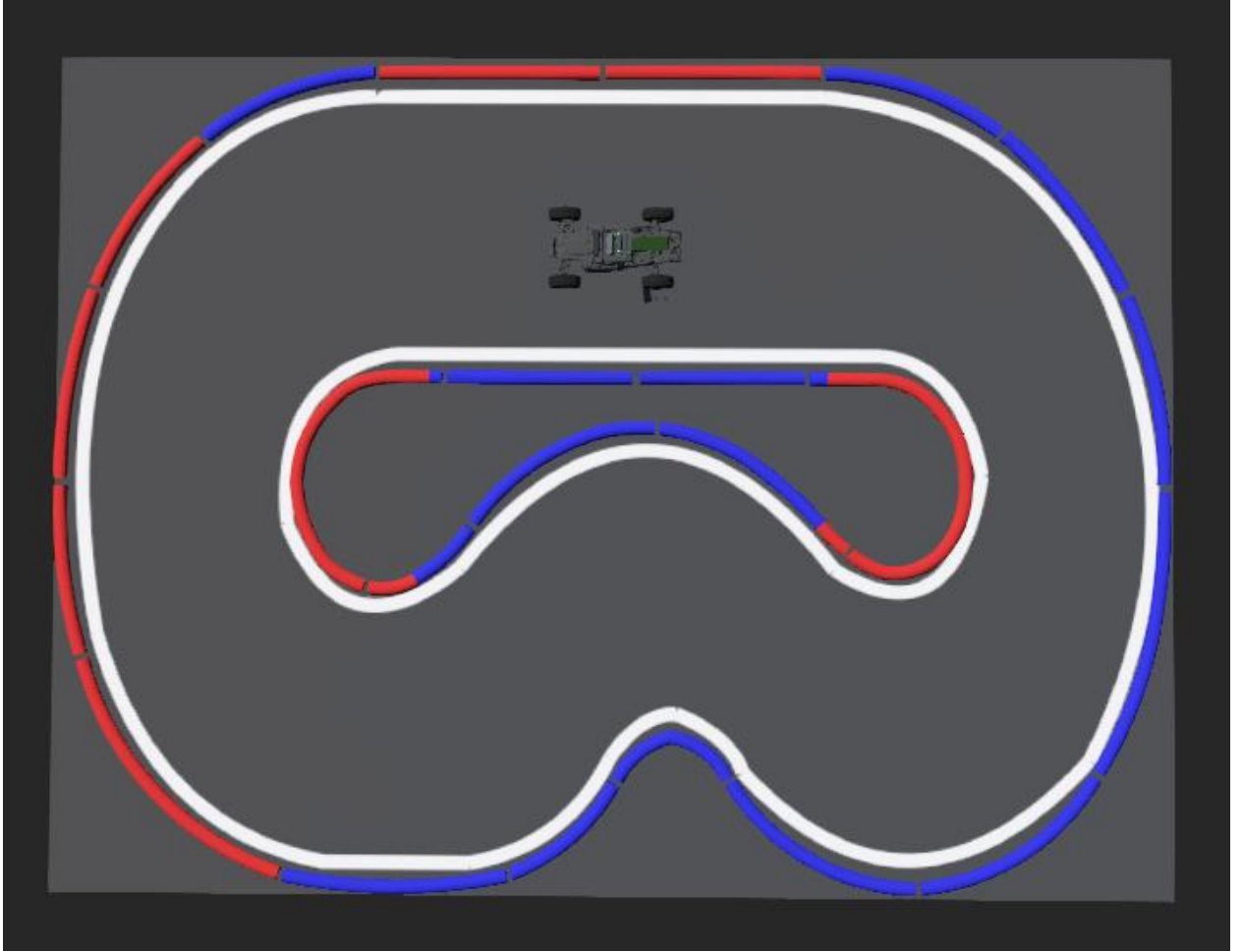


Figure 1: Final Webots racing environment used for simulation-based reinforcement learning.

5.2 Observation Space

The reinforcement learning agent utilizes visual sensor input as its main observation type. The majority of this sensor input is obtained by onboard sensors using visible light (RGB) cameras. The RGB camera is positioned on the front of the simulated vehicle. This is similar to how a camera is installed on the vehicle itself. The camera is mounted directly at the front of the vehicle, capturing what is in front of it, i.e., the track ahead.

As noted previously, the camera image size was decreased based on which algorithms were being tested. In addition, some of these tests utilized different ways of processing the images prior to their use by the agents. For example, the last PPO test used 64 x 64 pixel images that had been stacked into a single image of four frames, whereas, the SAC test used 160 x 80 normalized RGB images that had been stacked into a single image of two frames. These methods provided a lower computational cost than testing full resolution images, while still

providing enough detail to allow the agents to follow the track. The camera was placed approximately 18-19 cm above the ground, looking straight ahead with a slightly tilted view toward the bottom to provide a view of the track and the boundary wall.

Optional additional inputs available include inertial measurement unit (IMU) data. While generally used to determine angular velocity and acceleration of the vehicle, when combined with visual inputs, can assist the reinforcement learning agent in understanding more about the vehicle's motion and possibly to prevent potential oversteering situations.

All of the evaluation experiments have been focused solely on visual control as a means to assess the efficacy of using end-to-end reinforcement learning to learn a purely visual representation of control.

While both RGB and IMU/depth sensors are capable of providing much more detailed information regarding the vehicles state, all of the reinforcement learning evaluations conducted in this thesis used only RGB data as input to the agent. Additional sensor modalities such as IMU/depth will be explored in future work as extensions to current results.

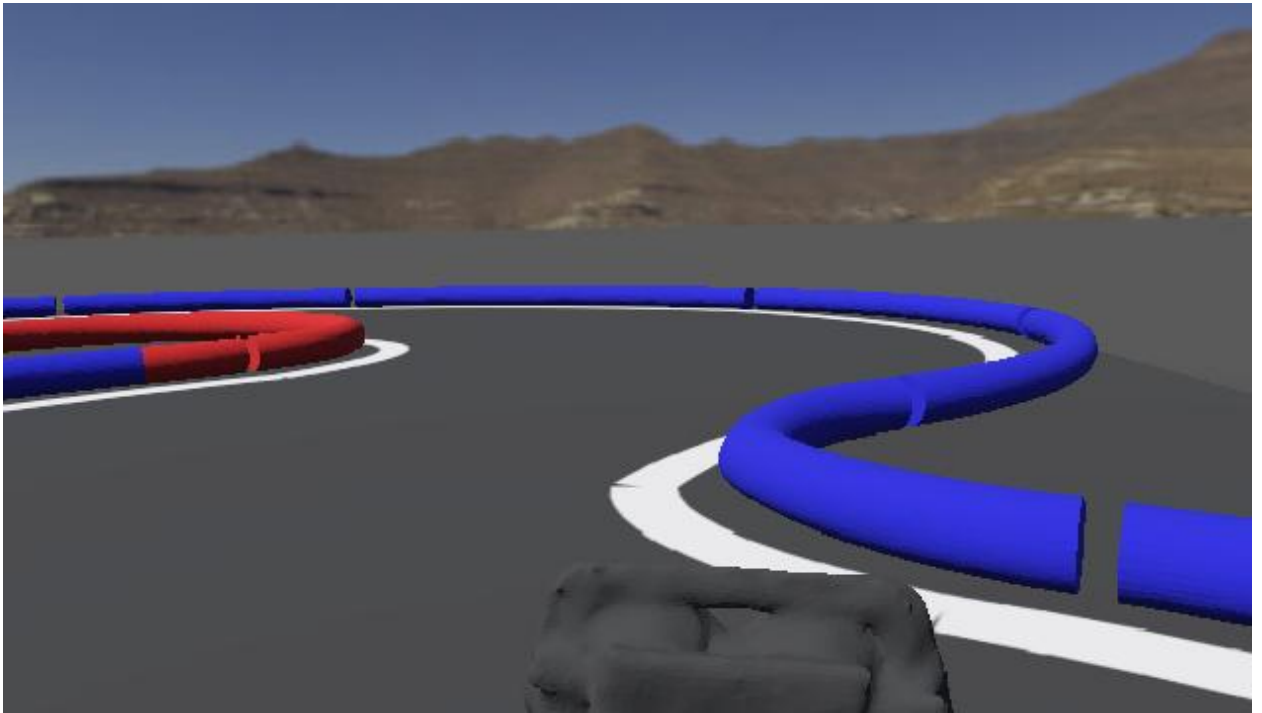


Figure 2: Example RGB camera observation used as input to the reinforcement learning agent.

5.3 Action Space

The control commands generated by the reinforcement learning agent consist of two continuous actions:

- steering angle
- throttle command

The steering action controls the front wheel angle via simulated steering servo, while the throttle action controls the motor through a simulated electronic speed controller (ESC). Both actions are represented as continuous values within predefined ranges.

The continuous action space reflects the control interface used by the physical vehicle, where steering commands are transmitted to the servo motor and throttle commands are sent to the ESC controlling the brushless motor.

Using continuous control allows reinforcement learning algorithms to learn smooth driving and optimize steering and acceleration during cornering.

5.4 Reinforcement Learning Algorithms

Two reinforcement learning algorithms are considered for this project:

- Proximal Policy Optimization (PPO) and
- Soft Actor-Critic (SAC).

PPO is used as the main long-run visual policy training configuration, while SAC is used as a comparative continuous-control method and later provides the strongest clean deterministic evaluation result.

5.5 Training Pipeline

The reinforcement learning training pipeline connects the Webots simulation environment with the Stable-Baselines3 reinforcement learning framework. The goal of the pipeline is to train a policy that works in simulation and to preserve a structure that can later be transferred to the physical vehicle with minimal redesign. For this reason, the simulator is exposed

through a Gymnasium-compatible interface, where the environment provides `reset()`, `step(action)`, `observation`, `reward`, and `termination` logic [20].

At each timestep, the policy receives a camera-based observation from the simulated front-facing RGB camera. The image is preprocessed into a compact format suitable for convolutional neural network processing. The policy then outputs continuous control actions, which are mapped to steering and motor commands inside the Webots vehicle model. After applying the action, the simulator advances one step, the reward is computed, and the next observation is returned to the learning algorithm.

The final training pipeline contains two reinforcement learning configurations. The first configuration is based on Proximal Policy Optimization (PPO), which is used as the main long-run visual policy training experiment. The final PPO environment is implemented through a dedicated `BuggyEnv` class. Before training, the environment is checked for compatibility using the `Stable-Baselines3` environment checker. It is then wrapped using `DummyVecEnv` and `VecFrameStack`, stacking four consecutive RGB frames so that the CNN policy receives short-term temporal information about vehicle motion.

The PPO model has been implemented using the `CnnPolicy` from `Stable-Baselines3`. It was then trained on the environment for 1000000 timesteps. The learning parameters were set so that the `learning_rate = .0003`, `n_steps = 2048`, and `batch_size = 64`. Training also produced Tensorboard log data with the `run_name: PPO_vision_run_1`, and the final checkpointed model was named: `ppo_buggy_final`.

In addition to the PPO implementation, we have implemented a comparison agent in the form of Soft Actor Critic (SAC). The SAC pipeline is an example of how one could implement a continuous control algorithm for a vision-based task. The observation space for the SAC environment consists of RGB images which are first resized to 160 x 80 pixels. Then each channel is reordered to be `channels_first` format, they are all normalized to the range `[0, 1]` and finally stacked into 2 frame stacks. The action space is defined as 2 continuous values representing throttle and steering. To help scale the steering values so that they do not exceed `max_steering_angle` when applying them to the vehicle’s steering mechanism, and to prevent “zero-throttle” from occurring during training, we constrain throttle to be greater than some small value. For both pipelines, a lightweight MobileNet-

like CNN based visual policy is employed to reduce the compute requirements of the policy, thereby making it more deployable as a real-time inference solution.

All three pipelines utilize CNNs to represent their visual policies since the input for each is image-based. During training we save out model checkpoints, tensorboard logs, and the evaluation metrics (reward, episode_length, policy_stability), to enable us to analyze performance at different stages of training.

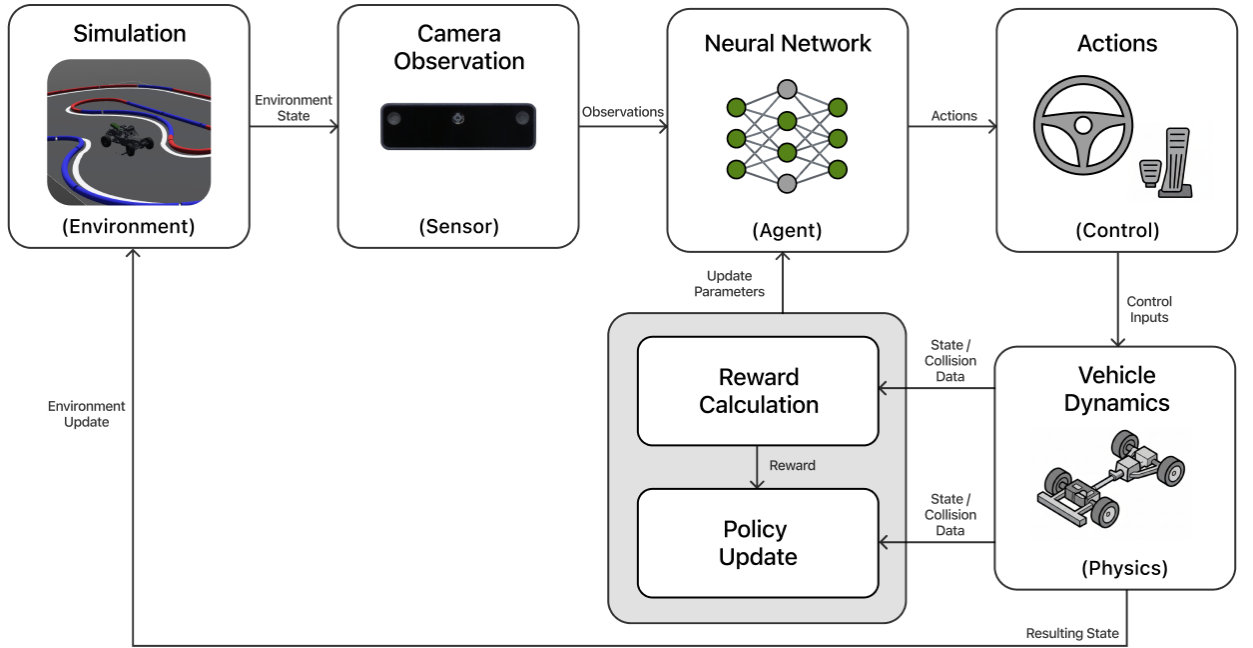


Figure 3: Reinforcement Learning training pipeline connecting the Webots simulation environment with the learning algorithm

5.6 Evaluation Metrics

In order to evaluate the effectiveness of the trained models we use two types of evaluation metrics. First evaluation is during training, we monitor and report training-time metrics that indicate if an improvement in performance has been made and how long the agent can stay on the road before it fails. These metrics include: mean episode reward, mean episode length, training stability indicators, and evaluation reward. We also monitor our evaluation metric during training so we know what type of rewards our model will produce at the end of training.

Second evaluations is during the deterministic post-training evaluation phase, the primary metrics reported are: episode length, total episode reward, average reward per step, termination reason, steering magnitude, throttle behavior, and action saturation. Due to unreliability of the lap counter in our final evaluation logs we do not provide an exact number of laps completed based upon those logs. Episode duration, max-step completion, red/blue boundary failures, and termination reasons serve as our primary indicators of driving stability.

The most important evaluation indicators are:

- mean episode reward during training,
- mean episode length during training,
- number of episodes reaching the maximum step limit,
- number of red/blue boundary failures,
- average reward per step,
- mean absolute steering command,
- steering saturation rate,
- throttle behavior for SAC,
- qualitative stability during Webots and real-world testing.

This evaluation approach makes the results more reliable because it avoids overstating lap-completion performance when lap counting was not explicitly implemented in the final logging pipeline. Instead, the analysis focuses on measurable behavior that is directly available from the training and evaluation logs.

6. SYSTEM DESIGN AND IMPLEMENTATION

This section will describe the total system design that has been developed and deployed for the autonomous miniature racing platform. Two parts are included with this design: a simulated training environment and a physical robot on which the real world deployment would take place. The architecture is designed to enable reinforcement learning policies to be trained in simulation and later transferred to the real vehicle.

The system integrates multiple subsystems including perception, decision making, and low-level vehicle control. Reinforcement learning operates as the decision component and receives input from sensors and generates output to send to the vehicle.

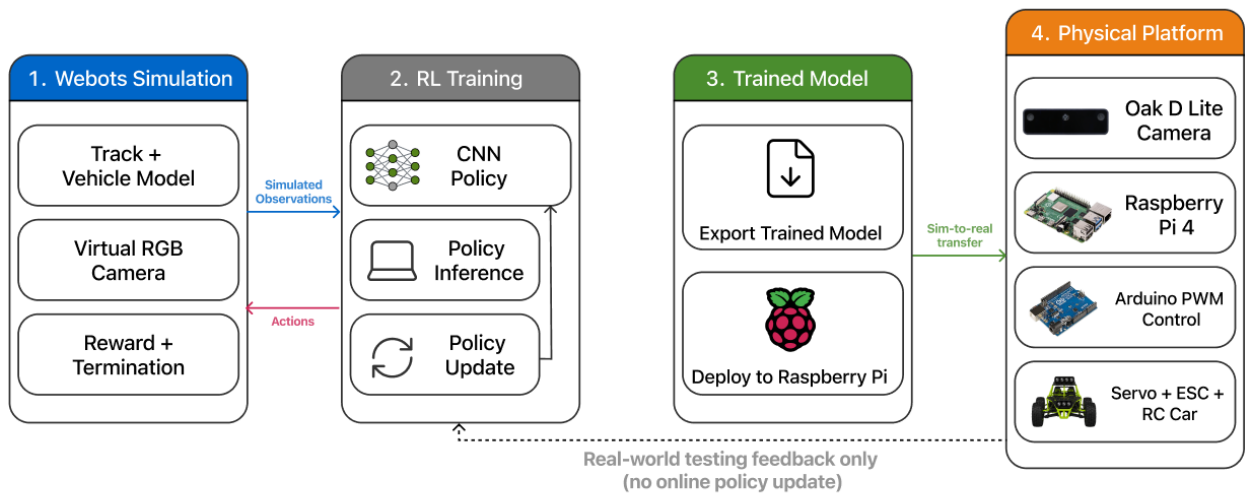


Figure 4: Overall architecture of the autonomous miniature racing system.

6.1 Hardware Platform

The modified RC Off-Road Buggy was chosen as a basis for the physical test platform due to its ability to support a large number of onboard computers/sensors that will be needed to accomplish autonomous testing. It provides a small, but very powerful base upon which to attach the necessary hardware to allow for autonomous operation.

The dimensions of the buggy are approximately 420mm long, 245mm wide, 180mm high, and it has a wheelbase of approximately 253mm. A brushless electric motor powered by an

Electronic Speed Controller (ESC) allows for precision throttle control over the motors throughout use.

Additional hardware components were added to the platform of the vehicle in order to allow for autonomous testing. The primary onboard computer is a Raspberry Pi 4 model, and it is responsible for executing the reinforcement learning policy and processing all data from sensors on board.

The perception system consists of a forward-looking RGB camera attached to the front of the vehicle. The RGB camera captures pictures of what lies directly ahead of the vehicle and supplies most of the observational information to the reinforcement learning agent.

The Raspberry Pi is not able to send the actual PWM (Pulse Width Modulation) signal to the actuators. Instead, the Raspberry Pi sends the commands for the steering and throttle using a USB Serial connection with an Arduino. The Arduino then generates the PWM signal to control the steering servo and the Electronic Speed Controller that controls the Brushed Motor. The separate functions of both boards will allow the Raspberry Pi to be used for Camera Processing and Neural Network Inference and the Arduino will provide low level timing for the Actuators.

.



Figure 5: Physical RC vehicle platform with the OAK-D Lite camera mounted on the front/top section for forward-facing visual perception.

Component	Role in System	Important Notes
RC buggy platform	Physical vehicle base	Modified miniature RC car used for autonomous racing experiments
Brushless motor + ESC	Throttle and propulsion	Controlled through calibrated ESC PWM signal
Steering servo	Front wheel steering	Servo center calibrated around 1520 μ s
Raspberry Pi 4	Onboard compute	Runs camera processing and model inference
OAK-D Lite camera	Visual perception	Provides RGB camera input for the RL policy
Arduino Uno	Low-level actuator interface	Generates stable PWM signals for steering servo and ESC
External actuator power	Servo/ESC power	Keeps actuator power separated from logic/control power

Table 1: Hardware components used in the physical autonomous racing platform.

6.2 Software Architecture

The overall design of the software architecture for the system has three major levels: the simulation and training environment, the reinforcement learning framework, and the onboard control system. The simulation level is written in Webots, the reinforcement learning algorithms are created using Stable-Baselines3, and the onboard implementation layer is also written in Python for the Raspberry Pi.

The Webots simulation environment is wrapped as a Gymnasium-compatible reinforcement learning environment. This interface enables the learning algorithm to communicate with the simulator via a common `reset()` and `step(action)` interface. The `reset()` function resets the simulation state and starts a new episode. The `step(action)` function runs the current control order, steps through the simulation, calculates the reward, and returns the next frame from the camera.

The last training environment for PPO is named “BuggyEnv”. BuggyEnv utilizes the Webots Supervisor API to interact with the Webots’s vehicle, cameras, steering inputs, and drive

motors. The observation space is represented by the compact RGB frame (resized) from a 640x480 camera at 64 x 64 pixels. The policy has 4 sequential image stacks in order to allow the CNN policy some form of short term memory (temporal context), which would enable it to determine if the vehicle was approaching a boundary or center itself on the track.

PPO's final action space is two-dimensional and continuous. The first action represents the amount of steer that the vehicle will turn and the second action represents how much power (speed/throttle) will be applied to the vehicle. The steering action is converted into front wheel angle utilizing the Ackerman Steering Approximation. The same type of turning geometry utilized to convert steering commands into left and right wheel speeds during cornering. This allows for a more real-world like simulation when compared to having the same speed applied to each wheel.

An approximation for a steering delay was added to the final PPO environment. In this case the desired steering (steering target) does not immediately move to that value. Instead it will eventually get there. As such, the "desired" steering value represents a gradual movement towards the target as opposed to instantaneous application which would be representative of the real world. By simulating the time it takes for the steering servo to respond we have taken another step in reducing the difference between what can be done in the simulation and what can be accomplished physically through actuation.

SAC training environment has a similar Webots-to-Gymnasium structure, however it utilizes a much less resource-intensive observation pipeline. The camera image is resized to 160x80 pixels, normalized, reshaped into channel first format and then two frames are stacked together. SAC policy generates both steering and throttle values. However, throttle is limited to a lower bound during training so that agents cannot learn strategies that do not allow them to make movements on the physical platform.

The physical vehicle's architecture mirrors the simulation pipeline. The physical vehicle contains an OAK-D Lite Camera providing RGB images as inputs. These RGB images are preprocessed and passed into a neural network residing on a Raspberry Pi for processing. Based on the results of the neural network process, the Raspberry Pi sends the resultant control values to an Arduino based low level interface controlling actuators. The Arduino converts these control values into PWM signals that send power to both a steering servo and

an Electronic Speed Controller. As such, this vehicle forms a closed loop system allowing continuous tracking of its current position, determination of an action based on those observations and finally sending that resultant action directly to the physical platform in order to apply the required controls.

Module/File	Environment	Main Responsibility
BuggyEnv	Webots simulation	Final PPO Gymnasium environment with 64×64 RGB observations and 2D action space
TrackNavigationEnv	Webots simulation	SAC training/evaluation environment with 160×80 normalized RGB observations
train_ppo.py	Training	Trains final PPO model for 1,000,000 timesteps
train_sac2.py	Training	Trains SAC model for 200,000 timesteps with evaluation callback
RealCameraAdapter	Real vehicle	Converts OAK-D Lite camera frames into model-ready observations
CarController	Real vehicle	Maps normalized actions into calibrated servo/ESC PWM microseconds
Real SAC deployment script	Real vehicle	Loads SAC model and executes safety-constrained autonomous control

Table 2: Main software modules used in the project.

6.3 Real-World Deployment Pipeline

The real-world deployment pipeline was implemented to validate whether the simulation-trained policies could be executed on the physical vehicle platform. The real world physical vehicle uses a Raspberry PI 4 as its onboard computer, an OAK-D lite as the camera for perception, and Arduino-based actuator interface to allow for steering and throttle control.

The real world loop is based on the same high level design as the simulation loop. First, the Oak-D Lite takes a forward facing RGB picture of the track. The module RealCameraAdapter gets the most recent frame available, shrinks the frame to the size

required by the models, changes color format as needed, optionally normalize the images, or apply frame stacking as needed. This produces an observation with the same structure expected by the trained policy.

After preprocessing, the observation is passed to the trained reinforcement learning model on the Raspberry Pi. The model outputs continuous control actions. These actions are not sent directly to the physical actuators. Instead, they pass through a safety and calibration layer implemented in the CarController module.

The CarController maps normalized steering and throttle values into calibrated PWM microsecond commands. The steering servo uses a calibrated center around $1520\ \mu\text{s}$, while the ESC neutral or stop signal is approximately $1498\ \mu\text{s}$. The generated command is sent to the Arduino over a serial connection.

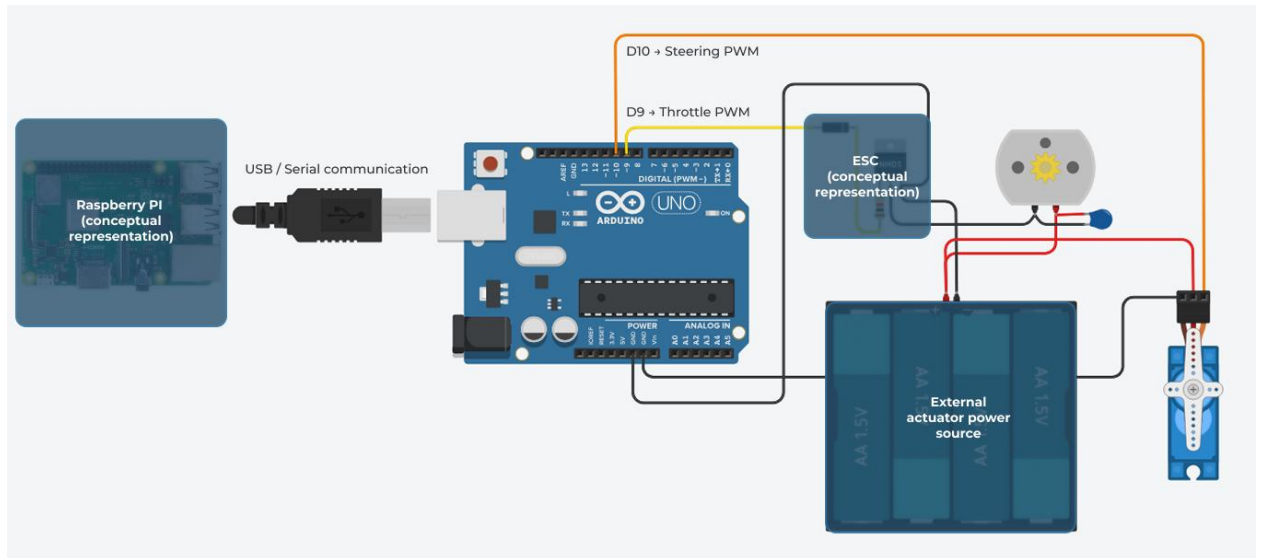


Figure 6: Physical control electronics architecture connecting the Raspberry Pi, Arduino, steering servo, ESC, and external actuator power.

Although the Raspberry Pi is capable of general-purpose input/output, it was not used as the direct PWM generator for the steering servo and ESC. Instead, the Raspberry Pi was responsible for high-level tasks such as camera processing, observation preprocessing, neural network inference, and safety logic, while the Arduino was used as a dedicated low-level actuator controller.

This separation was important for three reasons. First, the servo and ESC require stable microsecond-level PWM signals. Arduino is more suitable for this task because it is a microcontroller and can generate consistent timing signals without the scheduling variability of a Linux-based operating system. Second, separating high-level inference from low-level actuator control makes the system easier to debug. The Raspberry Pi sends simple serial commands, while the Arduino handles the physical actuator signal generation. Third, this design provides additional safety features since emergency stop and neutral commands are sent as simply as possible via a serial communications protocol that is easy to read and understand.

The communication protocol (i.e., how the Arduino communicates with the Raspberry Pi) was made simple. All of the commands contain a steering PWM value and an ESC PWM value in the format:

```
S <servo_us> E <esc_us>
```

During the last calibration, the steering servo middle or neutral position was about 1520 μ s, and the ESC neutral/stop value was about 1498 μ s. The calibrated PWM values for both actuators were then used by the CarController module to translate normalized model actions into actual control signals.

Manual arming, emergency stop, throttle limits, steering limits, enable/disable “boost,” smoothing of the steering angle and auto-disarm features will also be added to the scripts for controlling the robot in the real world due to these factors which require constraints.

In the SAC real world deployment script, the model output contains both steering and throttle. However, in the physical test phase, the throttle was constricted by a fixed-throttle safe operating strategy. Although the model throttle output was logged for later analysis purposes, the actual Electronic Speed Controller (ESC) command was restricted to minimize potential risks associated with uncontrolled vehicle acceleration. Thus, this was an example of a practical safety tradeoff made at an early point in time in order to facilitate successful sim-to-real validation.

.

6.4 Sim-to-Real Transfer Strategy

A key focus area of the sim-to-real transfer strategy is to maintain as much consistency as feasible in terms of the simulation pipeline vs. the physical deployment pipeline while also considering factors related to safety concerns and equipment capabilities. The main goal is to ensure that the trained policy receives similar visual observations and produces actions that can be mapped to the real steering servo and ESC.

The first strategy is observation consistency. In simulation, the agent learns from a front-facing RGB camera mounted on the vehicle. On the real platform, the OAK-D Lite camera is mounted in a similar forward-facing position. The camera frame is resized, converted, normalized, and stacked using the same general principles used during training. This reduces the mismatch between the simulated camera observation and the real camera input.

The second strategy is realistic action mapping. The final reinforcement learning environments use continuous steering and throttle-like actions. These actions are mapped to physically meaningful steering and motor commands rather than abstract movement commands. In simulation, Ackermann steering is used to convert a single steering action into separate left and right front-wheel angles. In real-world deployment, normalized steering values are converted into servo PWM commands using calibrated microsecond values.

The third strategy is actuator realism. The final PPO environment includes a first-order steering delay approximation, where the actual steering angle does not instantly match the target steering command. This mimics how long it takes for the actual steering servo to respond (i.e., delay) and lessens the difference between what you would ideally simulate as an action (ideal simulated actuation) vs. how your car acts in reality.

The fourth approach is safe real world adaptation. Due to the fact that the vehicle is very strong, the area we run on is relatively small, all deployments are done with limitations. Throttle is capped off, emergency stop is always active, and when the vehicle starts running autonomously, it is started by hand. By limiting these things, the policy learned can then be tested without causing harm to either the vehicle or sensors.

Even though this still doesn't remove every single thing about the sim-to-real gap. Ultimately there was a full pipeline to transfer everything: camera images from webots were used for training, models were saved and evaluated after being trained, a real OAK-D Lite adapter reproduced the image processing pipeline in order to produce identical pre-processed images, and a Raspberry PI to arduino interface mapped model outputs to commands for the actuators.

7. EXPERIMENTAL SETUP

This section describes the final experiment which used to train and evaluate reinforcement learning policies in Webots and to prepare the learned policies for real-world deployment.

7.1 PPO Training Configuration

The final PPO model was trained using the BuggyEnv Webots environment and the Stable-Baselines3 PPO implementation. The environment was first checked using `check_env()` to verify Gymnasium compatibility. It was then wrapped using `DummyVecEnv` and `VecFrameStack`, with four RGB frames stacked together to provide temporal visual context.

The `check_env()` function was used to verify that BuggyEnv followed the Gymnasium API correctly, including its observation space, action space, `reset()`, and `step(action)` behavior. `DummyVecEnv` wraps the environment in the vectorized interface expected by Stable-Baselines3, even when only one environment instance is used. `VecFrameStack` then stacks four consecutive RGB observations, allowing the CNN policy to receive short-term temporal information instead of only a single static image.

The PPO model used the `CnnPolicy` architecture. The base camera observation was resized to 64×64 pixels, and after four-frame stacking the model received a 12-channel image input. The action space consisted of two continuous actions: steering and speed/throttle-like motor command.

The PPO model was trained for 1,000,000 timesteps using a learning rate of 0.0003, `n_steps=2048`, and `batch_size=64`. TensorBoard logging was enabled during training, and the final model was saved as `ppo_buggy_final`. The saved model metadata reported 1,001,472 timesteps, which is consistent with PPO rollout collection slightly exceeding the requested training budget due to batching.

7.2 SAC Training Configuration

The SAC model was trained as a secondary continuous-control method. The SAC environment used RGB observations resized to 160×80 pixels, converted to channels-first format, normalized to the range $[0, 1]$, and stacked over two frames. The action space consisted of steering and throttle, with steering in the range $[-1, 1]$ and throttle in the range $[0, 1]$.

To avoid a degenerate solution where the agent outputs near-zero throttle and fails to explore the environment, throttle was clamped to a minimum value of 0.25 during training. The steering action was scaled by a maximum steering angle of 0.8 rad. Ackermann steering was used to calculate front wheel angles, and the motor velocities were adjusted according to the turn radius.

The SAC model used a lightweight MobileNet-like CNN feature extractor. Training was performed for 200,000 timesteps with a replay buffer size of 20,000, `learning_starts=1000`, `train_freq=8`, `gradient_steps=4`, and `ent_coef='auto_0.5'`. Evaluation was performed every 25,000 steps using an `EvalCallback`, and the best model was saved separately from the final model.

Parameter	Final PPO	SAC
Algorithm	PPO	SAC
Policy	CnnPolicy	CnnPolicy
Environment	BuggyEnv	TrackNavigationEnv
Observation size	64×64 RGB	160×80 RGB
Frame stack	4	2
Model input	12-channel stacked image	6-channel stacked image
Action space	Steering + speed command	Steering + throttle
Learning rate	0.0003	0.0003
Timesteps	1,000,000	200,000
Feature extractor	SB3 CNN policy	MobileNetLite
Reward style	Vision-centering + speed + steering smoothness	Distance progress + throttle/steering penalties
Main evidence	TensorBoard curves + deterministic 10-min run	EvalCallback + deterministic 10-min run

Table 3: Comparison of final PPO and SAC training configurations.

7.3 Reward Function Design

The final reward design differs between PPO and SAC because the two environments were developed for different training configurations.

The final PPO environment used a dense vision-based reward. A region of interest in the lower part of the camera image was analyzed to detect the road center and track boundaries. Red and blue boundary pixels were treated as failure indicators. Grey road pixels were used to estimate the center of the road in the camera image. The reward encouraged forward speed while keeping the road centered and penalized sudden steering changes.

The PPO reward can be conceptually summarized as:

$$R_t = 1.5 \cdot v_t \cdot (1 - e_t) - 2.0 \cdot |\Delta a_{steer}| \quad (3)$$

where v_t is the forward speed, e_t is the normalized visual center offset, and $|\Delta a_{steer}|$ is the change in steering action between consecutive timesteps. Here R_t represents the reward at timestamp t . e_t is normalized which means that it does not use millimeters, centimeters or similar stuff, rather it represents the visual offset as a scaled value between the detected road center and the image center. The Webots version includes a variety of physical quantities like distance and time. Distance is measured using meters to represent physical units. Time is similarly defined in terms of simulation units (seconds). The rewards are simply scaled versions of the numerical learning signals and do not measure anything physical. When the vehicle hit boundary of the track, or when vehicle get physically stuck, the vehicle receive very large terminal penalty.

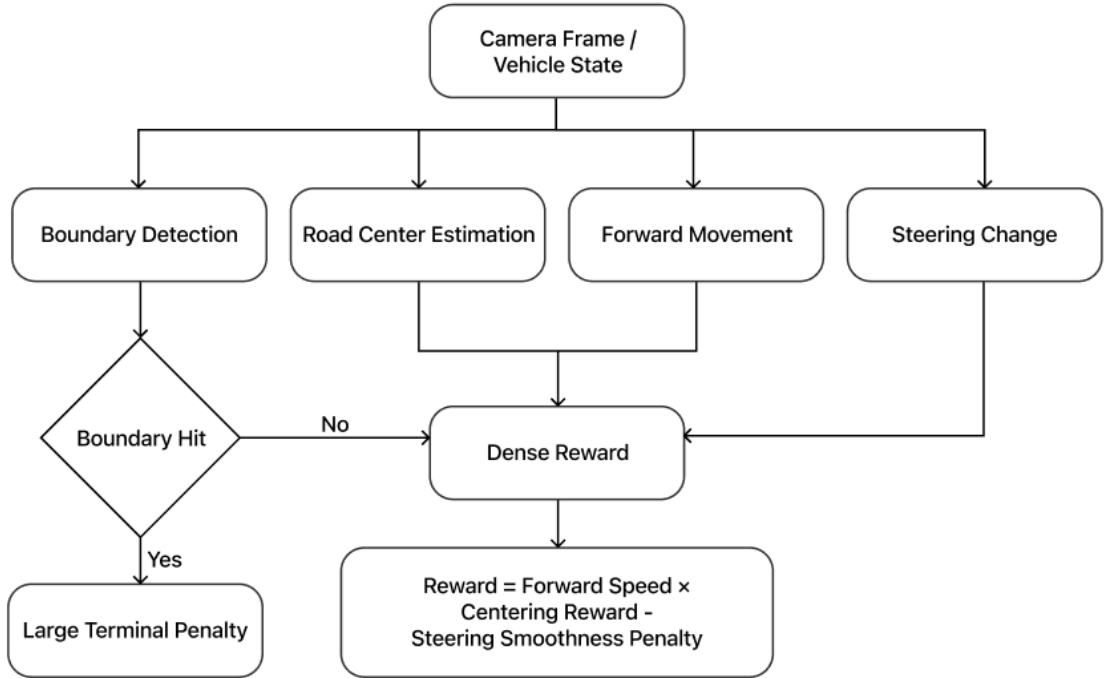


Figure 7: PPO reward design showing boundary detection, road center estimation, dense reward calculation, and terminal penalties.

The SAC reward was based primarily on distance travelled between consecutive timesteps. The reward was calculated as a scaled progress term plus a small survival reward. Additional penalties were applied for very low throttle and excessive steering. Terminal penalties were

applied for boundary contact, leaving the valid simulation area, becoming stuck, or exceeding the maximum episode duration.

The SAC reward can be conceptually summarized as:

$$R_t = 150 \cdot d_t + 0.5 - P_{throttle} - P_{steering} \quad (4)$$

Where R_t is the reward at timestamp (t), and d_t is the distance travelled between two consecutive timesteps. In the Webots simulator, the distance is in meters, and time is in seconds according to the simulator timestep. However, the reward itself is not a physical unit, but a scaled numerical signal used to guide learning. $P_{throttle}$ is applied when the throttle command is below 0.3, and $P_{steering}$ is applied when the absolute steering command exceeds 80% of the maximum steering angle. Terminal rewards override this dense reward: reaching the target zone gives a positive terminal reward, while boundary contact, leaving the valid world area, becoming stuck, or reaching the maximum episode length produce terminal penalties.

In the implementation, $P_{throttle} = 1.0$ when throttle < 0.3 , and $P_{steering} = 0.5$ when $|\text{steering}| > 0.8 \cdot \text{max_steering_angle}$.

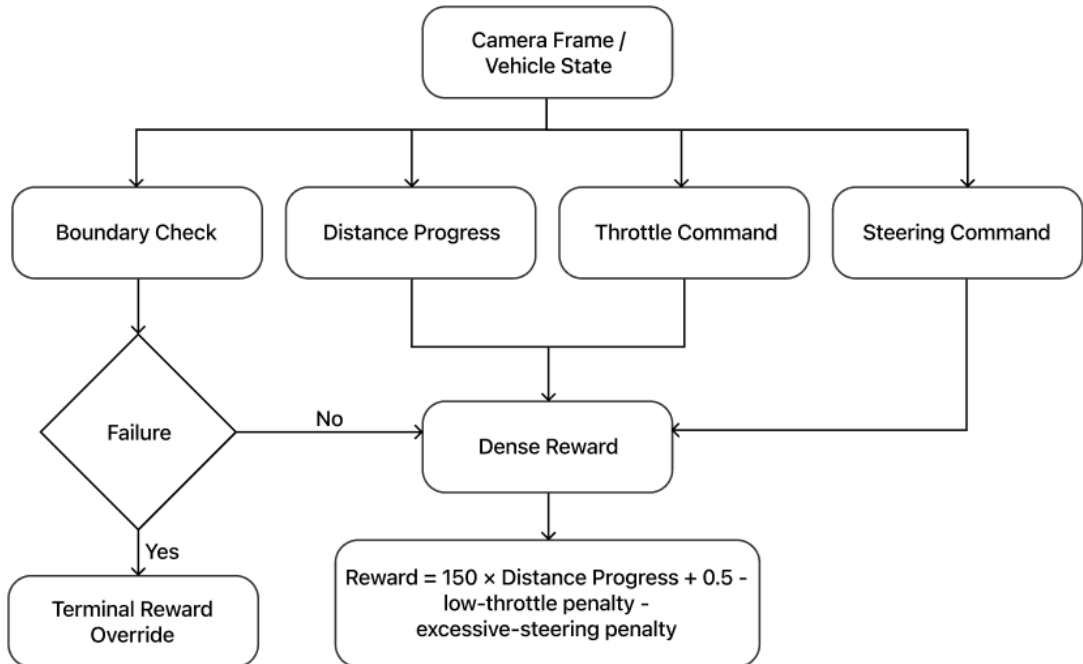


Figure 8: SAC observation and reward pipeline based on RGB input, distance progress, action penalties, and terminal safety conditions.

7.4 Evaluation Setup

The models were evaluated using both training logs and deterministic post-training evaluation. PPO training progress was evaluated using TensorBoard logs, including mean episode reward and mean episode length, and PPO was also tested in a logged 10-minute deterministic Webots run. SAC was evaluated using periodic evaluation logs saved during training and an additional 10-minute deterministic Webots run.

The deterministic evaluation was performed with exploration disabled. This means that the policy selected its most likely action for each observation rather than sampling exploratory actions. The evaluation focused on episode length, reward, termination reason, steering behavior, throttle behavior, and action saturation.

Evaluation at the end doesn't make claims about complete lap counts by lap due to integration issues in the last log pipeline. It is using step max completion, boundary fails, duration of episodes, and rewards that are used as evidence of stability.

7.5 Transition to Real-World Testing

The trained policies from simulation testing, were deployed to the real world test platform. The tests were completed under safe conditions with reduced throttles, manual arming, and an emergency stop capability. These tests were considered valid deployment of safety constrained tests instead of a full racing quantitative test.

8. RESULTS AND DISCUSSION

This section provides final results from the simulations and deployments performed during this project. Results are organized as follows: simulation implementation, training and evaluation of PPO, training and evaluation of SAC, comparison between SAC/PPO, validation of real-world deployment, limitations and conclusion.

8.1 Simulation Environment Implementation

Functional webots simulator was developed for the autonomously operated miniature RC car racing platform. The simulator includes a controllable model of a 1:12 scale RC vehicle, a track environment for the race, front-facing RGB camera observations, steering and drive motor interfaces (for control of the virtual vehicle), and a reinforcement learning wrapper compatible with openai Gymnasium.

Vehicle parameters in the vehicle model were measured or estimated from the physical properties of the actual platform. Those included wheelbase, track width, tire dimensions, drivetrain layout, mass distribution approximation. The design of the simulated track was made to be reproducible in the physical world. It is compact for indoor use and has curved turns with red/blue boundary walls for visual contrast. These boundary colors also provided a means for vision-based failure detection during training and evaluation.

The simulation environment supports clean resets through the Webots Supervisor API and exposes the required reinforcement learning functions: `reset()`, `step(action)`, observation generation, reward calculation, and termination conditions. This allowed the environment to be used directly with Stable-Baselines3 for PPO and SAC training.

8.2 PPO Training and Evaluation Results

The PPO model was trained for 1,000,000 timesteps using the BuggyEnv environment. The saved model metadata reported 1,001,472 timesteps, which is expected because PPO collects rollout batches and may slightly exceed the requested training budget. The model used a 12-

channel visual input created by stacking four consecutive 64×64 RGB frames, and a two-dimensional continuous action space for steering and speed control.

The PPO training curves show that the agent improved over time. Mean episode reward increased from approximately -0.13 at the beginning of training to approximately 379.37 by the end of the run, with a peak value of approximately 546.61. Mean episode length increased from approximately 123.57 steps to approximately 170.31 steps, with a peak value of approximately 266.27 steps.

For deterministic evaluation, PPO was tested in a 10-minute Webots run with exploration disabled. The run lasted approximately 600.03 seconds and recorded approximately 17,559 control steps across 17 logged episodes. The best episode length was 1084 steps, the average reward per step was approximately 0.493, and the mean absolute steering command was approximately 0.651. The steering saturation rate was approximately 48.5%, indicating that PPO relied on more aggressive steering behavior during this evaluation.

These results show that PPO learned meaningful autonomous driving behavior and could run continuously during deterministic Webots evaluation. However, compared with SAC, PPO produced shorter episodes and more frequent extreme steering commands, suggesting that the PPO policy was less smooth and less stable in the final deterministic evaluation.

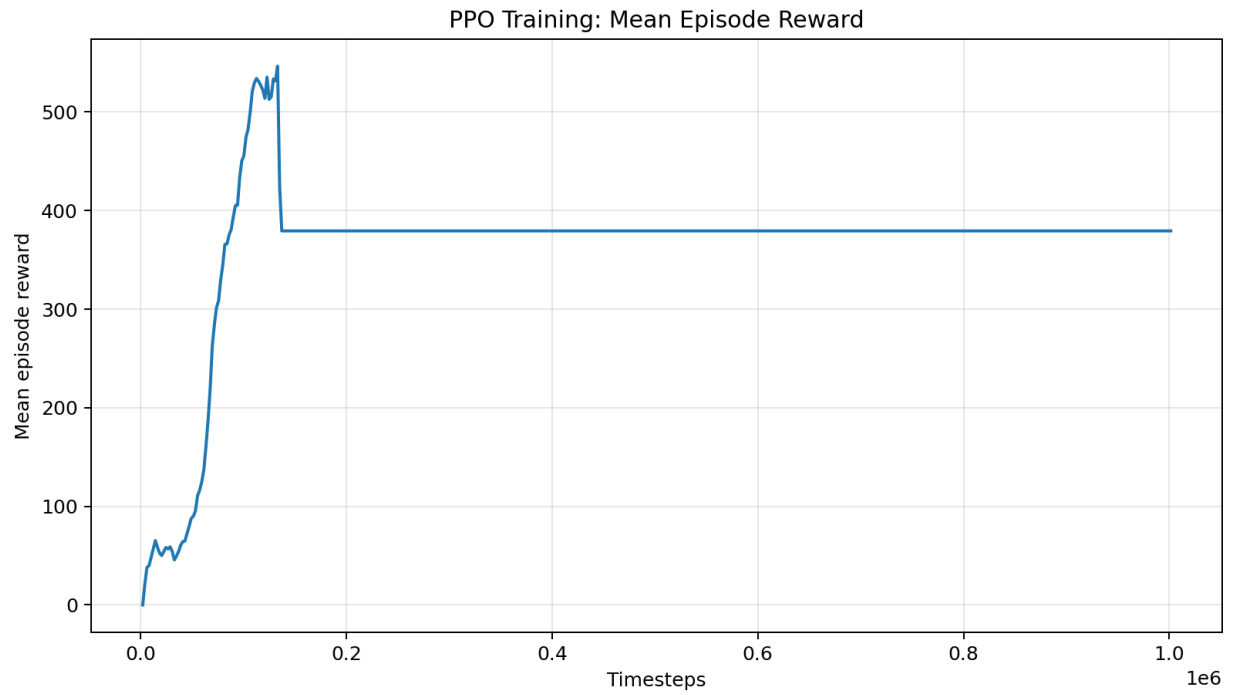


Figure 9: Mean episode reward during final PPO training. The curve shows learning progress over approximately 1,000,000 timesteps.

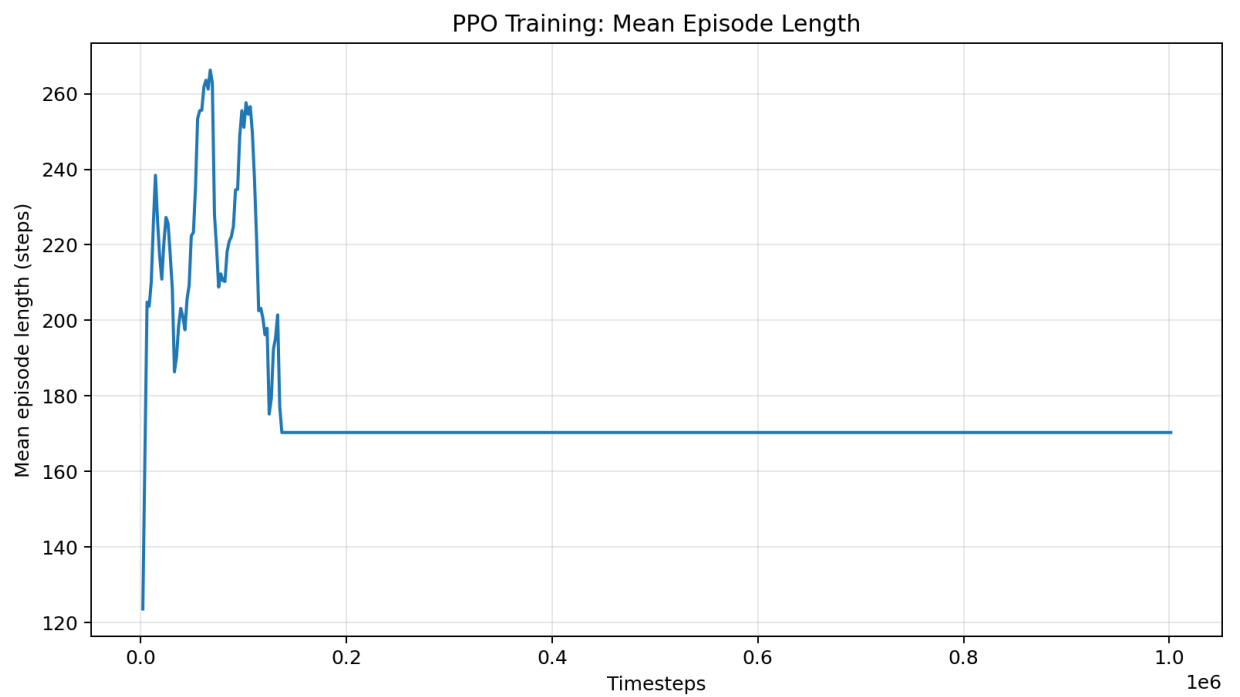


Figure 10: Mean episode length during final PPO training.

8.3 SAC Training and Evaluation Results

The SAC model was trained for 200,000 timesteps, with the best checkpoint saved at 175,000 timesteps. The SAC policy used a two-dimensional continuous action space consisting of steering and throttle. The observation consisted of two stacked RGB frames resized to 160×80 pixels and normalized to the range $[0, 1]$.

During training evaluation, SAC showed a strong improvement toward the later checkpoints. When I tested the model with an additional 20 hours of training, the top-performing model at timestep 175,000 had a mean evaluation reward of approximately 3766.18, and a mean episode length of approximately 1345.2 steps. In fact, four of five evaluations in which we used this model were able to complete all 1,500 steps before ending. This indicates that this model is quite stable for simulating long periods of autonomous vehicle operation.

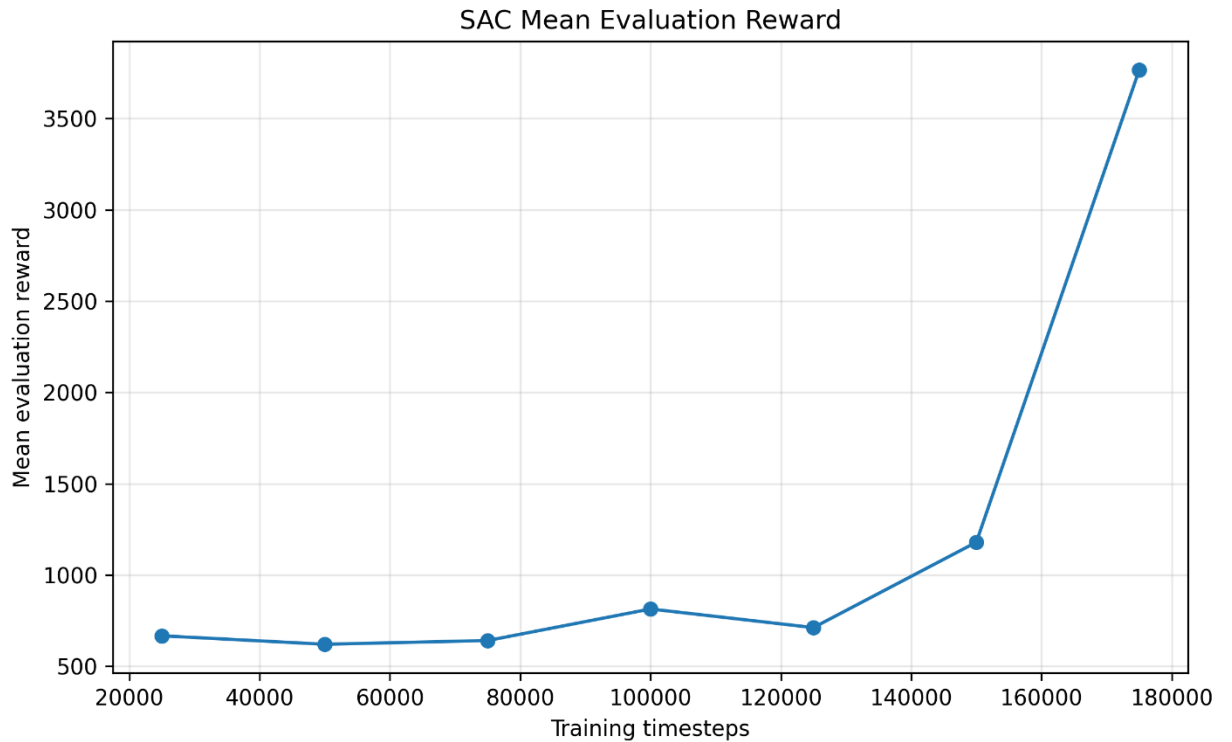


Figure 11: SAC mean evaluation reward over training timesteps. The strongest evaluation result occurred at 175,000 timesteps.

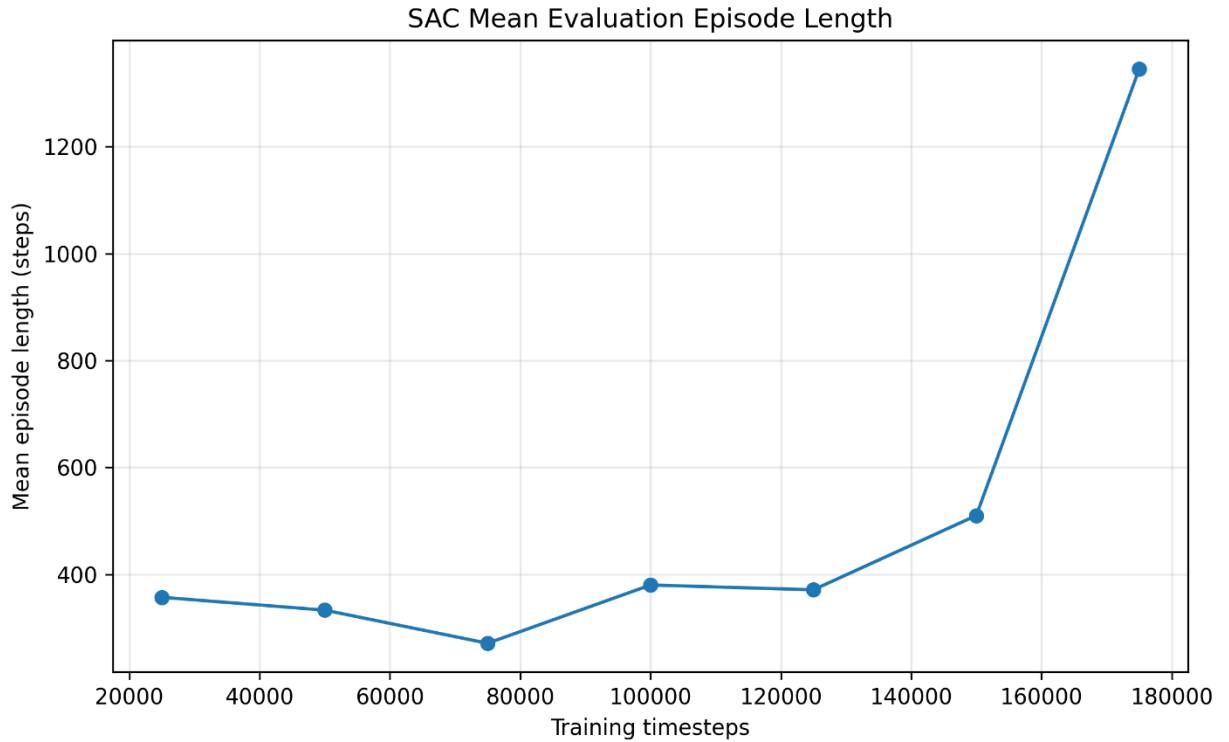


Figure 12: SAC mean evaluation episode length over training timesteps.

I also ran a 10 minute deterministic Webots evaluation to provide a more rigorous test of the model’s performance. During this evaluation, our agent executed approximately 16,297 control actions over the course of 13 logged episodes. Of these, ten episodes reached the maximum number of allowed steps (i.e., 1500), whereas two ended early because they encountered either red or blue boundaries, and one ended prematurely when the 10-minute time limit was hit.

Over this entire run, our agent averaged about 2.789 rewards per action taken, and our agent averaged roughly 0.80 on raw throttle values, and roughly 0.41 on absolute steering commands. Moreover, the steering saturation rate, the fraction of actions taken whose absolute steering command exceeded 0.8, was around 11.5%.

Overall, our data suggests that SAC can operate autonomously for long stretches of simulated time. However, our data also shows that our agent will fail occasionally. Specifically, we saw two instances where the agent failed to avoid red or blue boundaries. Thus, although SAC provides us with the strongest deterministic evaluation result from Webots in this project, it clearly has some significant limitations that would have to be resolved before deploying it aggressively into a real world environment.



Figure 13: Rolling SAC steering and throttle behavior during the 10-minute deterministic evaluation.

8.4 PPO and SAC Comparison

Both PPO and SAC ran fully autonomously on their own within the Webots simulation. PPO ran for an entire 10 minute period of time, demonstrating with the trained visual policy that it can control the vehicle autonomously by using only what is seen. Overall, however, SAC did better than PPO during deterministic evaluation.

SAC was able to reach its set maximum of 1500 steps in almost every episode it was logged, while the longest episode length for PPO as logged at 1084. In addition, SAC had a higher average reward per step (or action), a lower mean absolute steering value, and a lower steering saturation ratio. These values indicate that the steering output from SAC will be smoother and less erratic than PPO's steering when running in a deterministic mode.

Thus, PPO shows success in the training and operation of a long-running visual policy and autonomous inference, while SAC shows the greater deterministic evaluation. The comparison suggests that both algorithms were useful for the project, but SAC was more stable in the final evaluation setup.

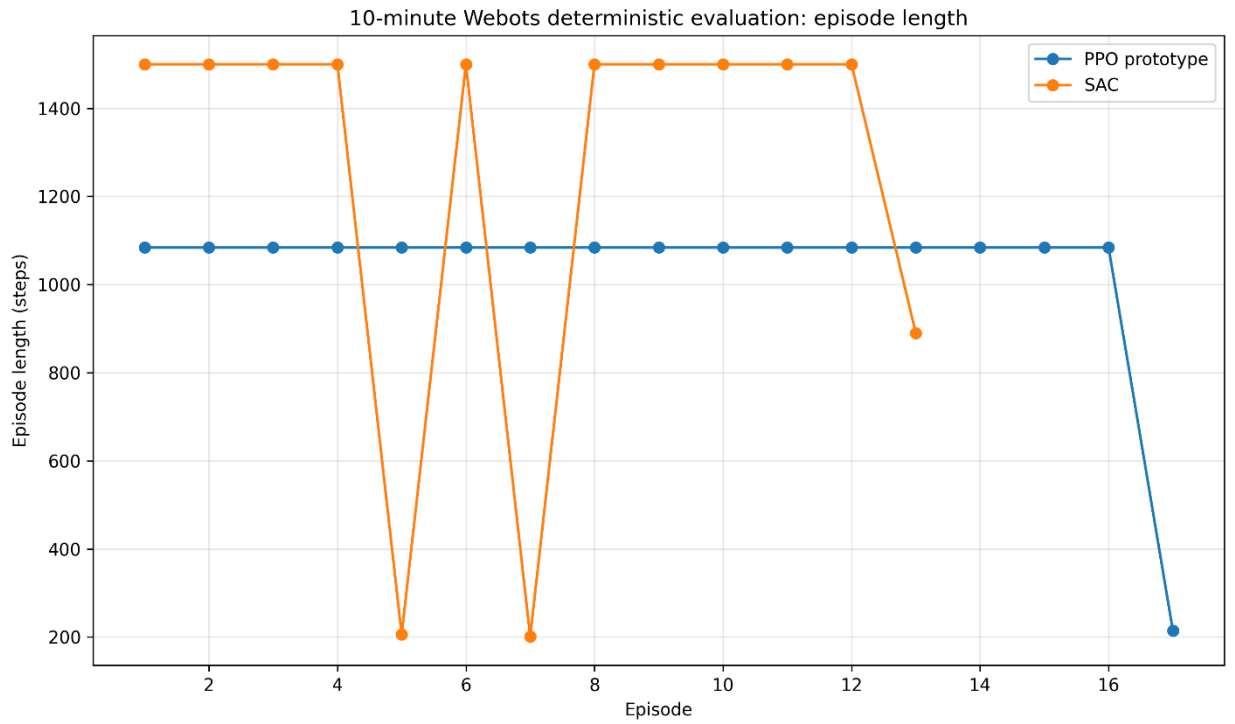


Figure 14: Episode lengths during 10-minute deterministic Webots evaluation.

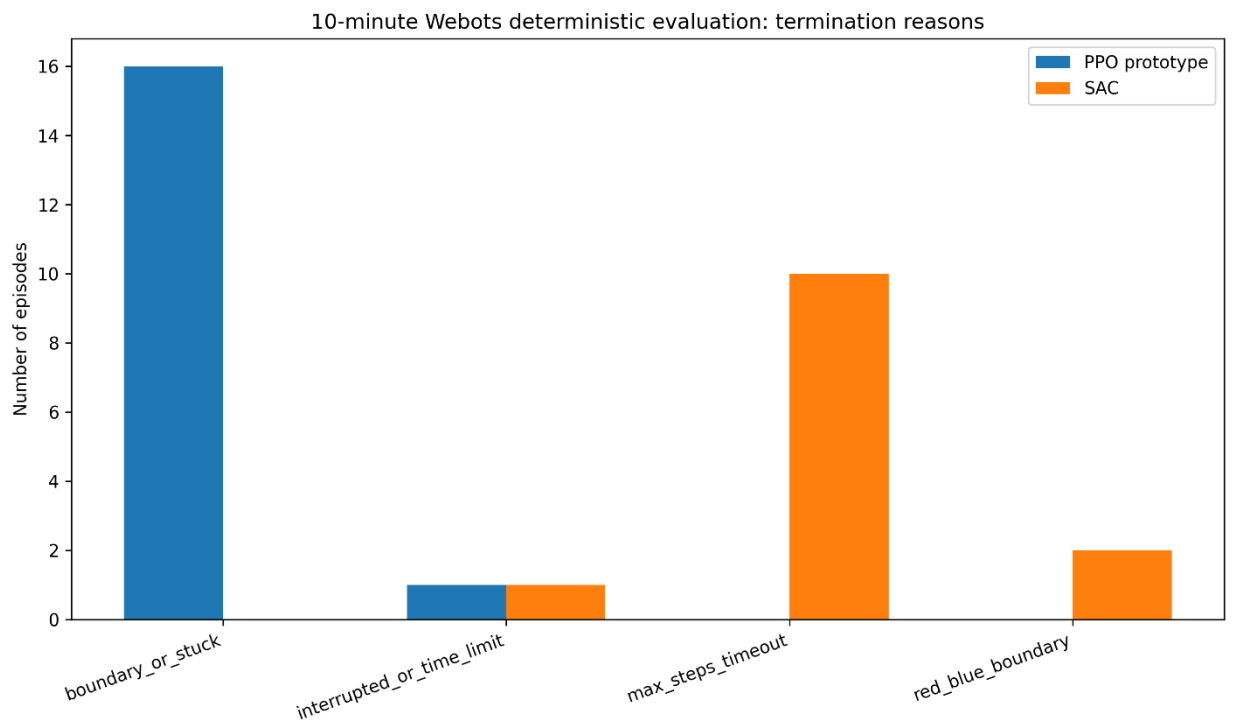


Figure 15: Termination reason distribution during deterministic Webots evaluation.

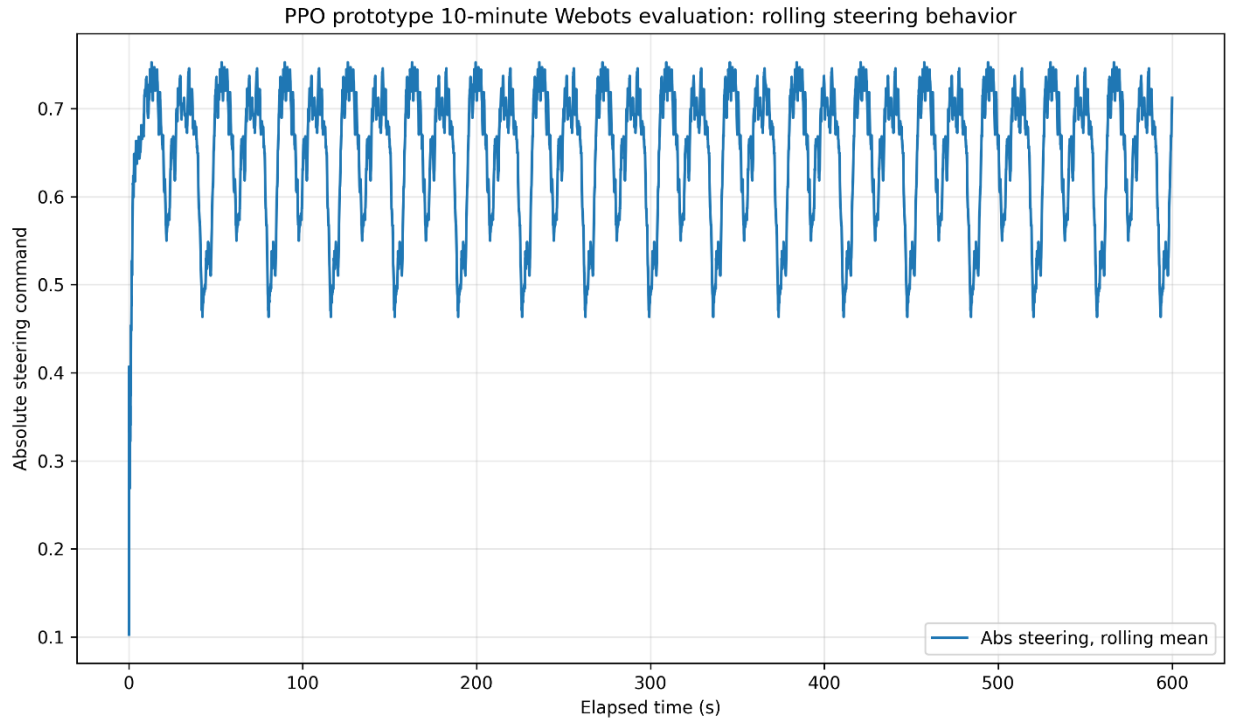


Figure 16: Mean absolute steering command per episode during deterministic Webots evaluation.

8.5 Real-World Deployment Results

The real-world deployment pipeline was implemented and tested at the code and system-integration level. The Raspberry Pi loads the trained model, receives camera frames from the OAK-D Lite through the RealCameraAdapter, preprocesses the observations, predicts actions using the reinforcement learning policy, and sends calibrated steering and throttle commands to the Arduino-based actuator controller.

The CarController module converts normalized steering and throttle values into PWM microsecond commands for the steering servo and ESC. The calibrated steering center is approximately 1520 μs , and the ESC neutral/stop signal is approximately 1498 μs . Commands are transmitted using a simple serial protocol of the form S <servo_us> E <esc_us>.

A real-world SAC deployment script was implemented with safety mechanisms including manual arming, emergency stop, maximum armed duration, throttle limiting, steering limiting, start boost, steering smoothing, and anti-wall throttle reduction. These safety

constraints were necessary because the physical vehicle is powerful, the track is small, and uncontrolled policy behavior could damage the vehicle or sensors.

During real world testing throttle was constrained manually instead of using the raw SAC throttle output. The model steering output was used as the primary learned control output and throttle was manually constrained for safe operation. Therefore, real world testing should be viewed as an initial sim-to-real validation of the trained policy integration into the physical vehicle control loop rather than a full validation of autonomous performance.

The training data indicated that real world visual conditions significantly impacted the behaviors of all the models. In real world testing, the SAC model generated the most promising results. Using cardboard surrounding to block out much of the viewable environment in the attic test area, the vehicle operated relatively smoothly and similar to the simulated model when tested on the attic track. The vehicle did not exhibit significant left-right oscillations. With the steering response set at nearly 1.0 (very little delay), the vehicle remained on the track for a few turns before requiring manual correction.

Additionally, during some of the better real world tests, it was noted that the SAC controlled vehicle would continue to operate around the track from three to four times (approximately) with minimal manual steering input. These observations were made within both the garage and attic test areas. However, due to the fact that there were very clean vertical wall boundaries along the entire length of the garage test area, it was generally necessary to provide greater amounts of manual steering assist compared to the attic area. The assistance was mostly a short steering correction to the right, rather than continuous manual driving or throttle control. This behavior suggests that the SAC policy had learned a meaningful track-following strategy, but still lacked the robustness required for completely independent operation in visually difficult sections of the physical track.

The main recurring failure occurred near the lower chicane before the final hairpin. This was also one of the visually difficult parts of the track in simulation. In the real environment, the vehicle usually passed the first part of the chicane but then failed to complete the middle section of the S-shaped turn and collided with the inner wall. This suggests that the camera observation was sufficient to initiate the maneuver, but not robust enough to consistently interpret the second part of the chicane under real lighting and wall-proximity conditions.

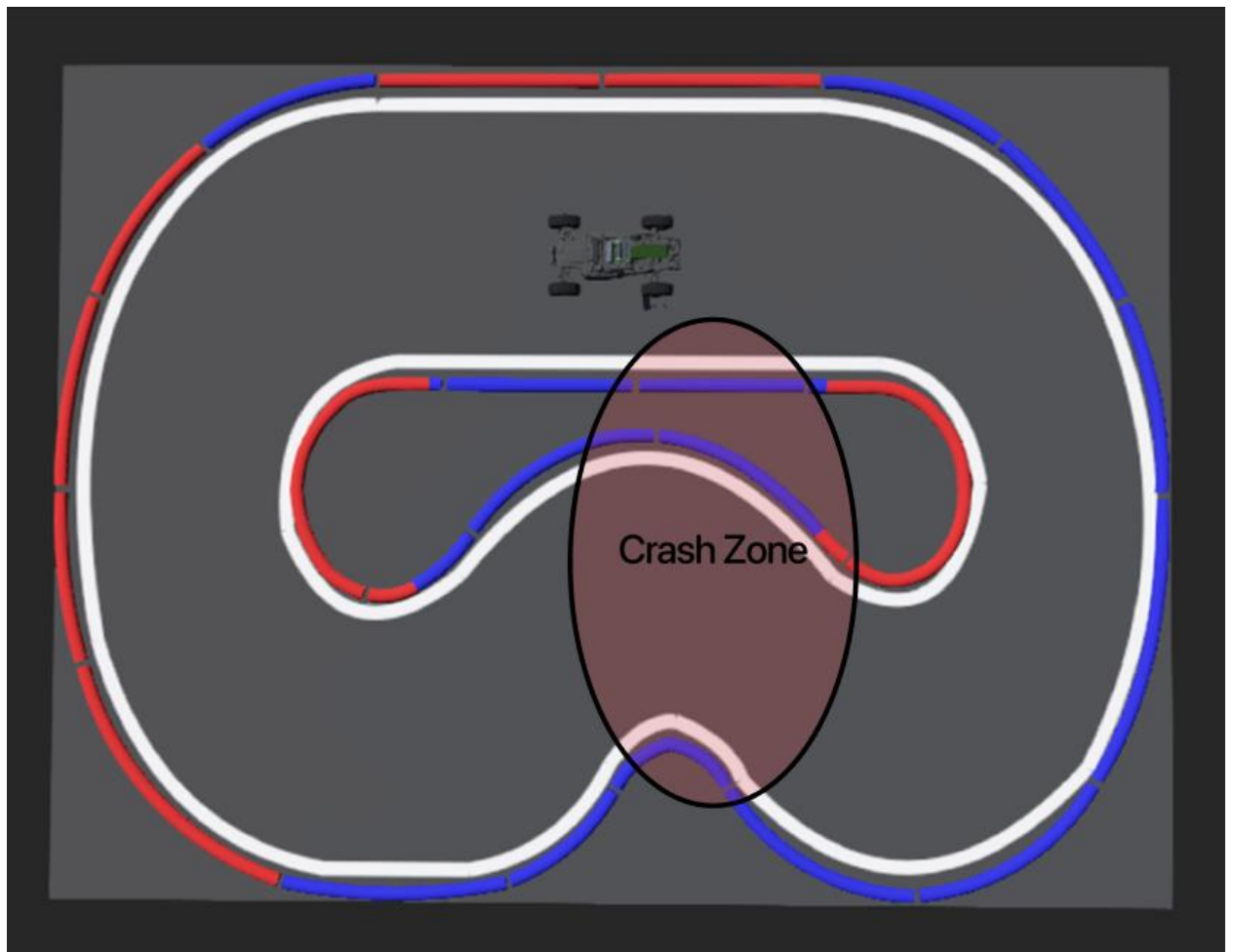


Figure 17: Problematic lower chicane crash zone observed during testing.

The OAK-D Lite camera pipeline was able to provide approximately 28–30 FPS during testing, which was sufficient for low-speed autonomous control. The primary cause of the observed delay came from viewing the preview using remote VNC as opposed to from streaming the recorded camera feed itself. Reviewing the recorded video immediately after testing was much smoother than reviewing the live VNC preview. This difference is significant since it indicates that the Visual Inference Pipeline (VIP) could be used. However, the Remote Monitoring Interface did not provide an accurate indication of real-time camera delays.

For the physical test, the throttling range with the least amount of instability was roughly 0.35-0.40. Values greater than this created excessive collision risks on the small indoor test track. A boost value of 0.65 for a very short duration was only useful to overcome initial static friction. Tuning of the steering response was also necessary. If the steering response

were set at low levels, the vehicle reacted too slowly. Levels close to 0.95-1.00 provided more reasonable correction behaviors in the physical world.

The steering response value of 0.95-1.00 is based upon the actual steering mapping used in the Autonomous Vehicle Script. Lower levels resulted in the vehicle reacting too slow, particularly when making tight turns. Levels near 1.00 produced better correction behavior.



Figure 18: Problematic track section observed during real-world testing.

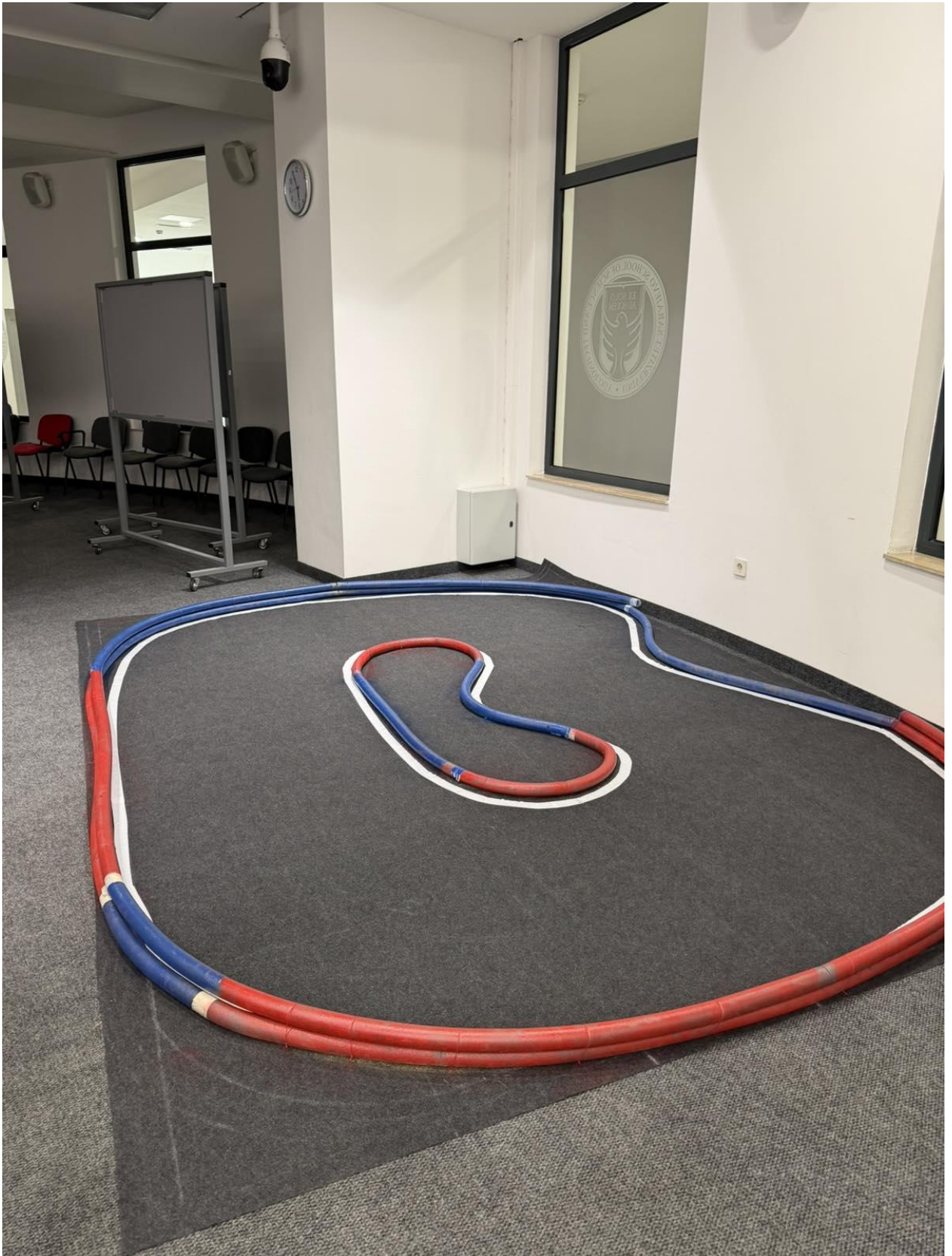


Figure 19: Physical indoor racing track used for real-world sim-to-real testing.

Another observed error occurred after the last hairpin when the vehicle occasionally turned into the turn too early at the beginning of the final turn (start-finish). It was believed that this

error was due to the camera's mounting position which resulted in it seeing the next curve before the vehicle physically saw it. Consequently, the policy would react based upon what it could see for its future path with respect to track geometry prior to the vehicle reaching the actual turning location.

The vehicle exhibited poor behavior in a garage testing environment than in an attic test area. The garage contained a track nearer clean vertical walls than the attic and also the vehicle would steer heavily to either the left or right. A possible cause of the strong steering action is that the camera-based policy recognized the contrast of the nearby wall or edge of the wall as part of the track boundary or white line as a lane marker and therefore steered away from it incorrectly. While adding dark color wall coverings somewhat lessened the visual distractions resulting in poor behavior, they did not completely eliminate them. Therefore, it appears that the proximity of the walls were a major real world discrepancy relative to how they were modeled in the simulator.

In terms of the real world, these tests demonstrated the ability for SAC to exhibit autonomous behaviors that are relevant in many ways. In fact, they show that an autonomous system can demonstrate useful capabilities when the real world conditions mirror those present during simulated experimentation. However, there are some areas where a driver is needed to assist or intervene with regards to controlling the vehicle. Thus, while the initial safety constrained sim-to-real results from this testing would be viewed as encouraging evidence, the total amount of autonomy demonstrated by the vehicle could not be characterized as a complete autonomous racing test.

The above observations provide insight into why the real-world transfer was most effective when the physical characteristics of the real world environment were closest to the characteristics used to create the simulated version. As well, the failure of the policy (the autonomous driving policy) did not occur at random. Rather, each failure was directly tied to specific differences in the physical environments relative to the simulated versions. These differences included wall proximity, how exposed to view certain parts of the track were, and/or what angle the cameras viewing the race were pointed. It is very important to recognize that this suggests that while there may have been challenges associated with the implementation of the autonomous policies, there are other challenges that are based on how

the environments were designed, how consistent sensor data has to be, and/or how the vehicle needs to be calibrated for proper operation.

8.6 Limitations

Several limitations existed with this project.

First, PPO and SAC were built using different settings for environments, observations, rewards, and CNN models. Thus, the comparison was useful in terms of engineering design. However, it would not be considered a completely engineered algorithmic test.

Second, PPO performance was shown by means of long-term training curve data, model metadata, and one logged deterministic Webots inference run, whereas SAC was shown by periodic results from the EvalCallback method and ten minutes of deterministic testing. Therefore, because each of these algorithms were trained differently along with having different reward structures the comparison will be viewed as an engineering comparison and not a completely engineered algorithmic test

Thirdly, in order for autonomous racing to become deployable in a “real world” environment, there were several safety concerns which needed to be addressed with limits on throttle and requiring the driver manually arm the vehicle. The need for safety was for the sake of protecting hardware. However, this meant that even though real world testing occurred, it was not an entirely unconstrained autonomous racing test.

One other practical concern of deploying autonomous vehicles is how to properly calibrate and arm the electronic speed controller (ESC). It has been shown through actual hardware testing that the steering servo will produce movement when commanded appropriately by the computer controlling the vehicle, but the ESC will not allow throttle input from the computer unless the neutral signal produced by the computer matches the previously determined neutral position of the ESC, or unless the battery voltage meets a predetermined threshold at start-up. Therefore, it is demonstrated here that in addition to the policy being used in learning to control autonomous vehicles, there are other critical requirements including proper initialization of actuators, accurate determination of neutral positions, and consistency in available power.

Another limitation was the limited amount of time, space, and physical resources for conducting real world tests on the experiments. The experiments were designed to be conducted in a controlled environment that provided consistent lighting, physical track building and charging of batteries with manual supervision. Because of the constraints related to the above factors the real-world evaluation was only able to be completed as a safety-constrained validation test instead of being used as a large-scale quantitative benchmark.

Finally, the sim-to-real is still another significant limitation of this research. There are differences in lighting exposure by cameras, friction, battery voltages, servo responses and other factors related to the physical race tracks that will be present when training policies in simulated environments.

8.7 Discussion

The research demonstrates that a full simulation to deployment reinforcement learning (RL) pipeline was successfully built. This project developed a fully functioning Webots simulated environment with Gymnasium compatible RL interfaces, trained PPO and SAC models, generated TensorBoard and CSV logs of evaluations, and built a complete pipeline for deploying the model on Raspberry Pi, OAK-D Lite, Arduino, servo and ESC control.

In terms of technology, this project's major findings are that image based RL can be used to develop meaningful autonomous vehicle behaviors within the Webots simulated environment. In addition, PPO clearly improved throughout its long term training session while SAC was able to provide consistent deterministic performance throughout the 10 minutes of the Webots evaluation.

The project demonstrated that sim-to-real transfer is based upon much more than saving a model that has been trained. Camera pre-processing pipeline, action mapping, actuator limitations, safety logic and hardware calibration are all important factors when deploying the real world system. The real-world test environment needed an extra layer of safety to limit the throttle control to be safe so that the learned policy could be tested responsibly.

In relation to the research question, the results show that the transfer was partially successful.

The learned policies were able to produce meaningful autonomous driving behavior in simulation, and the real-world deployment pipeline successfully executed camera-based inference and actuator control on the physical vehicle. The SAC policy also showed the most promising physical behavior. In the best observed real-world attempts, including the garage setup, the vehicle was able to continue for approximately three to four laps with occasional small manual steering corrections. The attic setup produced similar behavior and generally required less manual assistance because the visual surroundings were closer to the simulation and the walls were not as close to the track. This is not treated as a fully autonomous benchmark result, but it shows that the simulation-trained policy transferred beyond purely virtual testing and produced usable real-world steering behavior under controlled conditions.

A complete transfer (fully) did occur using the pipeline and controlled testing. But there were some physical variables (lighting, wall proximity, etc.) that degraded performance when they varied from those of the simulated environment. The garage setup had additional manual involvement due to being closer to clean vertical walls, which caused less of a problem with the visual distractions, than the attic setup. In summary, this study found that reinforcement learning policies trained on simulations could be transferred into a physical mini-racing car at the pipeline and controlled testing levels. However, an effective and reliable unconstrained racing experience would require further modification (of both the physical space and the sensors used), better environmental control, and improvements in sensor technology.

The research demonstrated the possibility of creating a complete sim-to-real RL pipeline for a small-scale autonomous racing car. However, it highlighted many challenges that prevented a completely accurate or successful transfer directly from simulation to physical equipment.

In terms of practical application one of the most significant takeaways from this project was how important the surrounding area around the track actually was. This same policy performed far better in the attic test environment where the visual environment was so close to what the simulated environment provided, then compared to the garage environment test where a large amount of visual distraction occurred from the clean walls near by. This suggests that future training should not randomize only the track surface and lighting, but also the surrounding environment around the track.

9. CONCLUSION

This thesis presented the design, implementation, and evaluation of a sim-to-real reinforcement learning pipeline for autonomous miniature racing. The project combined a Webots simulation environment, image-based reinforcement learning, trained PPO and SAC policies, and a real-world deployment stack based on Raspberry Pi, OAK-D Lite camera input, Arduino actuator control, steering servo, and ESC-based throttle control.

A functional Webots environment was developed around a miniature RC vehicle model and a compact indoor racing track. The environment provides camera-based observations, continuous control actions, Ackermann steering, reward calculation, and episode termination logic. This environment enabled the use of Stable-Baselines3 algorithms for reinforcement learning training.

The final PPO model was trained for 1,000,000 timesteps using a CNN policy and four-frame visual stacking. Training logs showed a clear improvement in mean episode reward and episode length over time, and the PPO policy was also evaluated in a logged 10-minute deterministic Webots run. SAC was trained as a secondary continuous-control method and achieved the strongest deterministic Webots evaluation result.

The real-world deployment pipeline was also implemented. The OAK-D Lite camera input was processed using a reusable camera adapter, model inference was executed on Raspberry Pi, and the resulting commands were mapped to physical servo and ESC PWM values through an Arduino-based control layer. This demonstrated the practical feasibility of transferring the simulation-trained policy pipeline toward a real vehicle platform, although reliable real-world autonomy still required safety constraints and manual supervision.

The central research question focused on how well an autonomous reinforcement learning policy developed in simulation would be able to perform on a real mini racing car. Overall, the work shows that it is possible for simulated policies to produce meaningful driving behaviors using vision-based inputs and deploy those in a pipeline with a physical deployment. At the same time, the findings from this study support the idea that sim-to-real transfer has significant challenges.

Eventually, as an indoor prototype, this project may expand to become a more generalized research test bed for miniature robotics autonomy using a variety of learning techniques. Future experiments can include the testing of sensor integration, safer forms of reinforcement learning (RL), on-line policy adaptation, and hybrid RL-rule based systems combining learned policies with rule based safety barriers. As such, this research will help to develop the autonomous vehicle technology beyond just the context of autonomous racing, and provide insight into how better to solve general sim-to-real transfer problems for small mobile robots.

10. FUTURE WORK

Future research will need to enhance both simulation and real world testing. Firstly, it will be important for us to develop an evaluation framework that is completely functional, includes accurate lap counting, accurate lap time measurement, accurate crash classification and accurately calculates success rates. This will provide strong basis for comparing PPO, SAC and future reinforcement learning methods.

Secondly, we need to enhance the ability of our model to generalize from simulation to real world through increased domain randomization [21]. We can do this by randomly changing lighting, textures, camera exposure, friction, vehicle mass and actuator delays in the simulation environment while training the policy so it is more adaptable to varying conditions present in the real world.

Third priority is advancement of observation space. In addition to RGB camera as a primary source for input in this thesis, there are other available features provided by the OAK-D lite (depth and imu) that may be used in future versions of a model which includes the combination of RGB, depth, and inertial observations to enhance vehicle stability particularly at high speeds around corners and/or when limited or no visual tracking information is present.

Fourth priority is real-world fine-tuning. Following collection of safe real world operating conditions from vehicles, it would be feasible to utilize imitation learning, offline reinforcement learning, residual policy adaptation to fine-tune policies with data collected from actual vehicle operation.

Another area of future improvement will be to integrate simple proximity sensors (time-of-flight or ultrasonic) around the vehicle. Real-world tests showed that nearby walls and visually confusing boundaries were two of the main causes of incorrect steering behavior for the camera-based policy. Instead of replacing the camera-based policy with proximity sensors, these could act as a lightweight safety layer detecting when the vehicle is too close to a wall. If so, reduce throttle. Reject unsafe steering commands. Trigger an emergency correction. This type of system will provide value in garage setups where nearby walls strongly affect the camera-based policy.

An enhanced form of sensor input is use of LiDAR. In contrast to the simplicity of proximity sensors, LiDAR provides a much greater geometric representation of the track and its perimeter wall. The ability for the system to generate an estimation of available free-space and reliably determine the edges of the track are two advantages that LiDAR has over other forms of sensor technology. Combining learning based visual control with rules based obstacle avoidance/localization methods could also be an effective way to utilize this sensor. It is worth to mention that adding a LiDAR module would increase the complexity of the system as well as the costs associated with it and the computational resources needed by the processor. Therefore, utilizing the least expensive means (proximity sensors) is most practical at this time in the development process while reserving the utilization of LiDAR for a later development phase.

Finally, the physical track and hardware platform should be refined further. A more consistent physical track surface, improved lighting, better camera mounting, and more precise actuator calibration would reduce the sim-to-real gap and allow more reliable real-world evaluation.

BIBLIOGRAPHY

- [1] J. Betz et al., “Autonomous Vehicles on the Edge: A Survey on Autonomous Vehicle Racing,” *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458-488, 2022, doi: [10.1109/ojits.2022.3181510](https://doi.org/10.1109/ojits.2022.3181510)
- [2] M. O’Kelly, H. Zheng, Dhruv Karthik, and Rahul Mangharam, “F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning,” *PMLR*, pp. 77–89, Aug. 2020, Accessed: Jan. 10, 2026. [Online]. Available: <https://proceedings.mlr.press/v123/o-kelly20a.html>
- [3] J. Kabzan *et al.*, “AMZ Driverless: The Full Autonomous Racing System,” *arXiv:1905.05150 [cs]*, May 2019, Available: <https://arxiv.org/abs/1905.05150>
- [4] A. Liniger, A. Domahidi, and M. Morari, “Optimization-based autonomous racing of 1:43 scale RC cars,” *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, Jul. 2014, doi: [10.1002/oca.2123](https://doi.org/10.1002/oca.2123).
- [5] R. Verschueren, S. De Bruyne, M. Zanon, J. Frasch, and M. Diehl, “Towards Time-Optimal Race Car Driving using Nonlinear MPC in Real-Time.” Accessed: Jan. 10, 2026. [Online]. Available: <https://publications.syscop.de/Verschueren2014a.pdf>
- [6] “Reinforcement Learning,” *MIT Press*. R. S. Sutton and A.G. Barto
<https://mitpress.mit.edu/9780262039246/reinforcement-learning/>
- [7] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic Policy Gradient Algorithms,” *proceedings.mlr.press*, Jan. 27, 2014.
<https://proceedings.mlr.press/v32/silver14.html>
- [8] T. P. Lillicrap *et al.*, “Continuous control with deep reinforcement learning,” *arxiv.org*, 2015. <https://arxiv.org/abs/1509.02971>
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv.org*, Aug. 28, 2017. <https://arxiv.org/abs/1707.06347>
- [10] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, and P. Duerr, “Super-Human Performance in Gran Turismo Sport Using Deep Reinforcement Learning,” *IEEE Robotics and Automation Letters*, pp. 1–1, 2021, doi: [10.1109/lra.2021.3064284](https://doi.org/10.1109/lra.2021.3064284).
- [11] Y. Song, H. Lin, E. Kaufmann, P. Duerr, and D. Scaramuzza, “Autonomous Overtaking in Gran Turismo Sport Using Curriculum Reinforcement Learning,” *arXiv.org*, 2021. <https://arxiv.org/abs/2103.14666>
- [12] M. Jaritz, de Charette, M. Toromanoff, E. Perot, and F. Nashashibi, “End-to-End Race Driving with Deep Reinforcement Learning,” *arXiv.org*, 2018.
<https://arxiv.org/abs/1807.02371> (accessed Jan. 10, 2026).

- [13] B. Evans, H. W. Jordaan, and H. A. Engelbrecht, “Comparing deep reinforcement learning architectures for autonomous racing,” *Machine Learning with Applications*, vol. 14, pp. 100496–100496, Dec. 2023, doi: [10.1016/j.mlwa.2023.100496](https://doi.org/10.1016/j.mlwa.2023.100496).
- [14] G. Czechmanowski, J. Węgrzynowski, P. Kicki, and K. Walas, “On learning racing policies with reinforcement learning,” *arXiv.org*, 2025.
<https://arxiv.org/abs/2504.02420>
- [15] J. Pan, J. Xing, R. Reiter, Y. Zhai, E. Aljalbout, and D. Scaramuzza, “Learning on the Fly: Rapid Policy Adaptation via Differentiable Simulation,” *arXiv.org*, 2025.
<https://arxiv.org/abs/2508.21065> (accessed Jan. 10, 2026).
- [16] F. Djeumou, M. Thompson, M. Suminaka, and J. Subosits, “Reference-Free Formula Drift with Reinforcement Learning: From Driving Data to Tire Energy-Inspired, Real-World Policies,” *arXiv.org*, 2024. <https://arxiv.org/abs/2410.20990>
- [17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” *proceedings.mlr.press*, Jul. 03, 2018.
<https://proceedings.mlr.press/v80/haarnoja18b.html>
- [18] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, pp. 1–8, 2021, Available:
<https://jmlr.org/papers/volume22/20-1364/20-1364.pdf>
- [19] O. Michel, “Cyberbotics Ltd. WebotsTM: Professional Mobile Robot Simulation,” *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, p. 5, Mar. 2004, doi: [10.5772/5618](https://doi.org/10.5772/5618)
- [20] G. Brockman *et al.*, “OpenAI Gym.” Available: <https://arxiv.org/pdf/1606.01540>
- [21] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World.” Available: <https://arxiv.org/pdf/1703.06907>